

Universidad de La Habana
Facultad de Matemática y Computación



El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares

Autor:

Jossué Arteche Muñoz

Tutores:

Msc. Fernando Raul Rodriguez Flores

Lic. Raudel Alejandro Gómez Molina

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

Mayo de 2026

A mi familia

Agradecimientos

A mis padres, por su apoyo incondicional en cada etapa de mi formación, por estar siempre a mi lado y por creer en mí en todo momento.

A mis tíos, por su presencia constante y por brindarme siempre su cariño. En especial a mi tía Dorita por su apoyo, y a mi tío Alfre por enseñarme a programar.

A mis abuelos, por ser mi mayor ejemplo de esfuerzo, dedicación y humildad.

A David, Yusbel, Javier, Daniela, Colominass y al resto de mis amigos de la Lenin, por ofrecerme una amistad verdadera e inseparable, y por tantos momentos compartidos que atesoro.

A mi novia, Eveline, por ser mi todo en estos últimos años, por su amor, su paciencia y su compañía incondicional en este camino.

A Diego, Francisco, Mauricio, Luis Alejandro, Pablo y al resto de mis amigos de carrera, con quienes compartí proyectos, estudios, alegrías y varios 2 en discreta.

A los profesores de la facultad, que con su saber y dedicación contribuyeron a mi formación profesional. De manera muy especial, a mis tutores Fernando y Raudel, por su guía constante, su paciencia y su valiosa ayuda durante la realización de esta tesis.

A todos, muchas gracias.

Opinión de los tutores

En este trabajo se aborda un problema clásico de la Ciencia de la computación, el problema de la satisfacibilidad (SAT), desde una perspectiva que vincula la teoría de lenguajes formales con la complejidad computacional. Se propone una construcción que utiliza autómatas finitos para representar las interpretaciones que satisfacen cada cláusula de manera independiente, para luego intersectarlas con un lenguaje que fuerza la consistencia de la interpretación global. Este enfoque permite reformular el problema de satisfacibilidad como el problema del vacío para un lenguaje específico.

Un resultado particularmente relevante de esta tesis es el estudio crítico de las gramáticas libres de contexto múltiples paralelas (pMCFG). Jossué no solo aplica formalismos existentes, sino que identifica un error en el procedimiento de intersección reportado en la literatura, lo que constituye un hallazgo científico significativo. Además, demuestra que el problema de intersección entre una pMCFG y un lenguaje regular es NP-duro, contribuyendo así a una mejor comprensión de los límites de este formalismo.

Como podrán comprobar las personas que lean este documento, Jossué tuvo que estudiar contenidos que no forman parte de su plan de estudios, incursionando en temas avanzados de teoría de lenguajes, autómatas y complejidad, y lo hizo de manera creativa y rigurosa. También demostró capacidad para identificar problemas abiertos o errores en la literatura existente y proponer soluciones originales.

Estamos muy satisfechos con la independencia demostrada por Jossué durante todo el proceso de investigación, la calidad de los resultados obtenidos, así como la profundidad del análisis teórico presentado.

Estamos en presencia de un trabajo excelente, realizado por un excelente científico de la computación.



MSc. Fernando Raul Rodriguez Flores
Facultad de Matemática y Computación
Universidad de La Habana.



Lic. Raudel Alejandro Gómez Molina
Facultad de Matemática y Computación
Universidad de La Habana.

Resumen

El problema de la satisfacibilidad booleana (SAT) es un problema NP-completo que consiste en determinar si existe una asignación de valores de verdad a las variables de una fórmula booleana que haga que la fórmula sea verdadera. La rama de la teoría de lenguajes formales es una rama de la ciencia de la computación que estudia los lenguajes formales y sus propiedades. El objetivo de este trabajo es vincular el problema de la satisfacibilidad booleana con la teoría de lenguajes formales para construir el lenguaje de las interpretaciones que satisfacen una fórmula.

En este trabajo, se propone un enfoque que consiste en construir un autómata finito que reconoce el lenguaje de las interpretaciones que satisfacen a cada cláusula independientemente, asumiendo que cada cláusula puede satisfacerse con una interpretación diferente. Luego, se intersecta dicho autómata con un lenguaje que obliga a que cada cláusula se satisfaga con la misma interpretación. Esto permite que el problema de la satisfacibilidad booleana pueda resolverse determinando si el lenguaje resultante es vacío o no.

A partir de esta construcción se demuestra que, para cualquier formalismo capaz de generar este lenguaje de repeticiones y cerrado bajo intersección con lenguajes regulares, al menos uno de los siguientes problemas es NP-duro: construir la intersección con un lenguaje regular o decidir el vacío del lenguaje resultante.

Posteriormente se estudia el caso de las gramáticas libres de contexto múltiple paralelas (pMCFG), un formalismo reportado en la literatura como cerrado bajo intersección con lenguajes regulares y con problema de vacío decidible en tiempo polinomial. Se muestra que el procedimiento de intersección propuesto en la literatura para este formalismo es incorrecto mediante un contraejemplo, y se demuestra que el problema de determinar la intersección entre una pMCFG y un lenguaje regular es NP-duro.

Abstract

The Boolean satisfiability problem (SAT) is an NP-complete problem that consists of determining whether there exists an assignment of truth values to the variables of a Boolean formula such that the formula evaluates to true. Formal language theory is a branch of computer science that studies formal languages and their properties. The objective of this work is to relate the Boolean satisfiability problem to formal language theory in order to construct the language of the interpretations that satisfy a formula.

In this work, an approach is proposed that consists of constructing a finite automaton that recognizes the language of interpretations that satisfy each clause independently, assuming that each clause may be satisfied by a different interpretation. Then, this automaton is intersected with a language that enforces all clauses to be satisfied by the same interpretation. This allows the Boolean satisfiability problem to be solved by determining whether the resulting language is empty or not.

From this construction, it is shown that for any formalism capable of generating this repetition language and closed under intersection with regular languages, at least one of the following problems is NP-hard: constructing the intersection with a regular language or deciding emptiness of the resulting language.

Subsequently, the case of parallel multiple context-free grammars (pMCFGs) is studied. This formalism has been reported in the literature as being closed under intersection with regular languages and having a polynomial-time emptiness problem. It is shown that the intersection procedure proposed in the literature for this formalism is incorrect by means of a counterexample, and it is proved that determining the intersection between a pMCFG and a regular language is NP-hard.

Índice general

1. Preliminares	5
1.1. Teoría de lenguajes	5
1.1.1. Conceptos, operaciones y problemas relacionados con lenguajes	5
1.1.2. Gramáticas y Jerarquía de Chomsky	7
1.1.3. Lenguajes regulares y autómatas	9
1.2. Complejidad computacional	10
1.2.1. Notación asintótica	10
1.2.2. Clases de problemas	11
1.3. El problema de la satisfacibilidad booleana	12
1.3.1. Definiciones	12
1.3.2. SAT como problema NP-completo	13
1.3.3. SAT desde la perspectiva de lenguajes formales	13
2. SAT y el problema del vacío para la intersección con un lenguaje regular	15
2.1. Codificación de las interpretaciones de una fórmula booleana como cadenas	16
2.2. Autómata para el lenguaje $L_{\text{ind}}(F)$	17
2.2.1. Automata cláusula	17
2.2.2. Autómata fórmula	23
2.3. $L_{\text{sat}}(F)$ a partir de la intersección con un lenguaje regular	26
3. Gramáticas libres de contexto múltiple paralelas	30
3.1. Definiciones	30
3.2. Problema del vacío para pMCFG y MCFG	35
3.3. Intersección de pMCFG y MCFG con lenguajes regulares	38
3.3.1. Intersección de MCFG con DFA	38
3.3.2. Intersección de pMCFG con DFA	41

4. Lenguaje de las interpretaciones que satisfacen una fórmula em- pleando gramáticas libres de contexto múltiple	44
4.1. $L_{0,1,d}$ como lenguaje de múltiple contexto paralelo	44
4.2. Resultados sobre la intersección entre pMCFG y lenguajes regulares .	51
Conclusiones y Recomendaciones	53
Referencias	55

Índice de figuras

1.1.	Autómata finito determinista que reconoce el lenguaje $\{aa\}$	10
2.1.	Autómata cláusula asociado a la cláusula $C = (x_1 \vee \neg x_2)$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$	19
2.2.	Autómata cláusula con separador asociado a la cláusula $C = (x_1 \vee \neg x_2)$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$	23
2.3.	Autómata fórmula asociado a la fórmula $F = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$	25
3.1.	Ejemplos de funciones de reordenamiento sobre el alfabeto $T = \{a\}$	31
3.2.	Gramática libre de contexto múltiple paralela que genera el lenguaje $\{a^{2^n} \mid n \geq 0\}$	34
3.3.	Gramática libre de contexto múltiple paralela que genera el lenguaje $\{www \mid w \in \{0,1\}^+\}$	35
3.4.	Gramática libre de contexto múltiple paralela con símbolos generadores y no generadores.	36
3.5.	DFA $\mathcal{A}$ que reconoce el lenguaje $\{aa\}$	42
4.1.	Gramática pMCFG $G_{0,1,d}$ que genera el lenguaje $L_{0,1,d}$	45

Introducción

El problema de satisfacibilidad booleana (*SAT* por sus siglas en inglés) es un problema fundamental en la ciencia de la computación y la lógica matemática. Dada una fórmula booleana, compuesta por variables booleanas y operadores lógicos como conjunciones, disyunciones y negaciones, el problema de SAT consiste en determinar si existe una asignación de valores verdaderos o falsos a las variables que haga que la fórmula sea verdadera. En 1971, Stephen Cook demostró que SAT es un problema NP-completo [6], lo que implica que es al menos tan difícil como cualquier otro problema en la clase NP. Esta propiedad ha llevado a que SAT sea un punto de referencia para la complejidad computacional y ha motivado el desarrollo de numerosas investigaciones para la búsqueda de métodos eficientes para resolverlo [2].

La teoría de lenguajes formales y autómatas es una rama de la Ciencia de la Computación que estudia los lenguajes formales [6]. Esta teoría proporciona un marco formal para entender la estructura y el comportamiento de los lenguajes, así como para clasificar los diferentes tipos de lenguajes según sus características. La aplicación de esta teoría abarca una amplia gama de áreas, incluyendo la compilación, el análisis de programas, el procesamiento de lenguajes naturales y artificiales y la teoría de la computabilidad [6].

En este trabajo se vinculan las dos ramas de la computación mencionadas, proponiendo un enfoque para resolver SAT utilizando la teoría de lenguajes formales y autómatas.

En estudios anteriores realizados en la facultad de Matemática y Computación de la Universidad de La Habana [1, 9, 11], se propone resolver variantes específicas del problema de satisfacibilidad booleana mediante el problema del vacío de varios tipos de gramáticas.

En este trabajo se generaliza la idea de usar el problema del vacío para cualquier variante, proponiendo un método independiente del orden de las variables. Esto garantiza que el método sea aplicable a cualquier fórmula booleana, lo cual era una limitación en los estudios anteriores, donde se requería un orden específico de las variables.

Para ello se propone la construcción del lenguaje de las interpretaciones que satisfacen una fórmula booleana dada, asumiendo que la fórmula se encuentra en forma

normal conjuntiva y que cada ocurrencia de variable es distinta. Esta suposición permite construir un autómata finito determinista de tamaño polinomial con respecto a la longitud de la fórmula. Adicionalmente, se define un lenguaje auxiliar $L_{0,1,d}$ que codifica la consistencia de las asignaciones entre distintas ocurrencias de una misma variable, forzando que estas compartan el mismo valor. La intersección de ambos lenguajes permite caracterizar exactamente el conjunto de interpretaciones que satisfacen la fórmula. De esta forma, la satisfacibilidad de la fórmula se reduce a decidir si el lenguaje resultante es vacío.

Las gramáticas libres de contexto múltiple (MCFG) y libres de contexto múltiple paralelas (pMCFG) [12] son formalismos de gramáticas desarrollados como una generalización de las gramáticas libres de contexto (CFG) [6], con el objetivo de proporcionar un modelo más expresivo para describir lenguajes.

En este trabajo se estudia el uso de MCFG y pMCFG para describir el lenguaje de las interpretaciones que satisfacen una fórmula booleana dada mediante la intersección con un lenguaje regular, así como la complejidad de decidir si dicho lenguaje es vacío.

Dado que las MCFG y pMCFG son cerradas bajo intersección con lenguajes regulares y se puede determinar si son vacías en tiempo polinomial [12], surge la siguiente pregunta científica: ¿Es posible describir el lenguaje de las interpretaciones verdaderas de una fórmula booleana con MCFG o pMCFG y decidir si este es vacío en tiempo polinomial?

En este contexto, el objeto de estudio de este trabajo es la caracterización formal de los lenguajes asociados a interpretaciones de fórmulas booleanas que las satisfacen y el análisis de su descripción mediante distintos formalismos de la teoría de lenguajes. En particular, se analiza cómo estos lenguajes pueden ser representados mediante MCFG o pMCFG, así como las condiciones bajo las cuales su problema del vacío puede ser decidido en tiempo polinomial.

El objetivo general de este trabajo es dada una fórmula booleana F , definir y construir el lenguaje de las interpretaciones verdaderas para F .

Para cumplir el objetivo general, se plantean los siguientes objetivos específicos:

- Estudiar el estado del arte referido a los formalismos de teoría de lenguajes y el problema SAT.
- Establecer una representación de las interpretaciones de una fórmula booleana como una cadena.
- Definir el lenguaje de las interpretaciones verdaderas de una fórmula booleana dada.
- Construir el lenguaje de las interpretaciones verdaderas de una fórmula booleana mediante la intersección de un autómata finito determinista y el lenguaje $L_{0,1,d}$.

- Construir el lenguaje de las interpretaciones verdaderas de una fórmula booleana utilizando gramáticas libres de contexto múltiple paralelas.
- Demostrar que, aunque el lenguaje de las interpretaciones que satisfacen una fórmula booleana puede ser descrito por una pMCFG, decidir si este lenguaje es vacío es un problema NP-duro.

Una de las principales contribuciones de este trabajo es demostrar que, dado que las pMCFG generan el lenguaje $L_{0,1,d}$, el problema de construir la intersección de una pMCFG con un lenguaje regular es NP-duro. Este resultado contrasta con afirmaciones presentes en la literatura donde se presentan propiedades de las pMCFG que afirman que este problema puede resolverse en tiempo polinomial.

Este trabajo se ha estructurado en 4 capítulos: en el capítulo 1 se presentan los principales conceptos y definiciones que serán utilizados en el resto de la investigación, incluyendo una introducción a la teoría de lenguajes formales y autómatas, así como una descripción detallada del problema de satisfacibilidad booleana (SAT). Además, se realiza un análisis de los trabajos previos relacionados con la aplicación de la teoría de lenguajes para resolver variantes del problema de SAT.

En el capítulo 2 se muestra como codificar el lenguaje de las interpretaciones verdaderas de una fórmula booleana mediante una cadena de símbolos. Posteriormente se define el lenguaje de las interpretaciones verdaderas y se propone una construcción del mismo utilizando la intersección de un autómata finito determinista y el lenguaje $L_{0,1,d}$. Para finalizar se demuestra que decidir si es vacía la intersección de cualquier formalismo que describa el lenguaje $L_{0,1,d}$ con un lenguaje regular es un problema NP-duro.

En el capítulo 3 se introducen las gramáticas libres de contexto múltiple (MCFG) y libres de contexto múltiple paralelas (pMCFG), se describen los algoritmos para decidir si el lenguaje generado por una MCFG o pMCFG es vacío y para construir la intersección de una MCFG o pMCFG con un lenguaje regular. Se demuestra que el algoritmo propuesto para calcular la intersección de una MCFG con un lenguaje regular no es generalizable a las pMCFG y se discute la dureza de este problema.

En el capítulo 4 se muestra como construir el lenguaje de las interpretaciones verdaderas de una fórmula booleana utilizando pMCFG y se analiza la complejidad computacional de esta construcción. Se demuestra que para toda fórmula booleana, existe una pMCFG que describe el lenguaje de sus interpretaciones verdaderas y que decidir si esta es vacía es un problema NP-duro.

Finalmente se presentan las conclusiones, recomendaciones, trabajo futuro y las referencias bibliográficas del trabajo.

Para la realización de este trabajo, se utilizó inteligencia artificial generativa con el fin de proponer nombres de variables, elementos matemáticos complementarios, títulos de algunos capítulos y transiciones entre secciones del documento. La opinión

del tutor presentada en este documento fue generada con modelos grandes de lenguaje, en particular DeepSeek. Para ello se usó el resumen de la tesis y las características personales del estudiante que resultaron relevantes para el desarrollo del trabajo. Para alcanzar el tono propio del tutor, se proporcionaron los mismos elementos (resumen de la tesis y características del estudiante) de opiniones del tutor de cursos anteriores como ejemplos.

Capítulo 1

Preliminares

En este capítulo se presentan las definiciones que se utilizan en el resto del trabajo. Se introducen alfabetos, cadenas y lenguajes, así como operaciones y problemas de decisión asociados a lenguajes formales. Posteriormente se presentan las gramáticas formales y los autómatas finitos. Finalmente, se introducen nociones de complejidad computacional y el problema de satisfacibilidad booleana. Estas nociones serán de utilidad para analizar el uso del problema del vacío en lenguajes formales para atacar el problema de la satisfacibilidad booleana.

1.1. Teoría de lenguajes

En esta sección se definen conceptos fundamentales teoría de lenguajes y formalismos para describir y reconocer lenguajes que servirán de base para el contenido de los capítulos y secciones posteriores.

1.1.1. Conceptos, operaciones y problemas relacionados con lenguajes

El concepto básico en teoría de lenguajes es el de alfabeto, a partir del cual se construyen las cadenas y los lenguajes.

Definición 1.1. *Un alfabeto, usualmente denotado como Σ , es un conjunto finito no vacío de elementos llamados símbolos.*

A partir de un alfabeto, se puede definir una cadena, que corresponde a una secuencia finita de símbolos de dicho alfabeto.

Definición 1.2. *Una cadena sobre un alfabeto es una sucesión finita de símbolos pertenecientes a ese alfabeto.*

Existe una cadena especial denominada *cadena vacía*, que no contiene ningún símbolo.

Definición 1.3. *La cadena vacía es aquella que no posee ningún símbolo y se denota por ε .*

Se denota por $|w|$ a la longitud de una cadena w . La cadena vacía ε tiene longitud cero.

Con estos conceptos se puede definir un lenguaje.

Definición 1.4. *Dado un alfabeto Σ , un lenguaje sobre Σ es cualquier conjunto de cadenas formadas por símbolos de Σ .*

Se denota por Σ^* al lenguaje de todas las cadenas posibles sobre un alfabeto dado, y por Σ^+ al lenguaje de todas las cadenas no vacías.

Entre cadenas se define la operación de concatenación, esta operación permite unir dos cadenas de manera secuencial.

Definición 1.5. *Sean x e y dos cadenas. La concatenación de x e y , denotada por xy , es la cadena formada por todos los símbolos de x seguidos de todos los símbolos de y .*

Si se aplica la concatenación de manera repetida sobre una misma cadena, se obtiene lo que se conoce como potencia o copia de la cadena.

Definición 1.6. *Dada una cadena w y un entero $n \geq 0$, la copia o potencia n -ésima de w se define inductivamente como:*

Caso base: $w^0 = \varepsilon$

Paso inductivo: $w^n = ww^{n-1}$, $n > 0$.

Por ejemplo, sea el alfabeto $\Sigma = \{0, 1\}$. Algunas cadenas sobre este alfabeto son 0, 1, 10, 1101 y ε . El conjunto de todas las cadenas que terminan en 0 es un lenguaje sobre Σ , al que pertenecen las cadenas 0, 10, 1100, 1010 y no pertenecen las cadenas 1, 1101 y ε .

Al ser los lenguajes conjuntos de cadenas, es posible definir sobre ellos operaciones de conjuntos como unión, intersección y complemento [6].

Para este trabajo, es de particular interés el «problema del vacío», el cual consiste en determinar si un lenguaje dado contiene alguna cadena o no.

Definición 1.7. *El problema del vacío consiste en determinar, para un lenguaje dado L , si $L = \emptyset$.*

En el capítulo 2 se reduce el problema de la satisfacibilidad booleana a determinar si el lenguaje de las interpretaciones que satisfacen una fórmula codificadas como cadenas es vacío o no. El problema de la satisfacibilidad booleana se define en la sección 1.3.

En la siguiente sección se introducen las gramáticas y la jerarquía de Chomsky, que las clasifica en diferentes tipos. Estas servirán de base para definir en el capítulo 3 las gramáticas libres de contexto múltiple paralelas, las cuales se utilizarán en el capítulo 4 para generar el lenguaje de las interpretaciones formulas booleanas.

1.1.2. Gramáticas y Jerarquía de Chomsky

Con el objetivo de describir lenguajes se introducen las gramáticas formales. A continuación se presenta su definición formal.

Definición 1.8. *Una gramática es un formalismo definido mediante una 4-tupla:*

$$G = (N, T, P, S)$$

donde:

- N es un conjunto de símbolos llamados símbolos no terminales.
- T es un conjunto de símbolos llamados símbolos terminales. Debe cumplirse que $N \cap T = \emptyset$.
- P es un conjunto de pares ordenados de la forma:

$$(\alpha, \beta)$$

tales que $\alpha \in (N \cup T)^* - T^*$ y $\beta \in (N \cup T)^*$ llamados reglas de producción.

Las reglas (α, β) se denotan como $\alpha \rightarrow \beta$.

- S es un símbolo no terminal llamado símbolo inicial de la gramática.

Una derivación en una gramática consiste en seleccionar una regla de producción $\alpha \rightarrow \beta$ y sustituir una ocurrencia de α en una cadena $w \in (N \cup T)^*$ por β [6].

Una cadena $w \in T^*$ se puede generar por una gramática G si existe una secuencia de derivaciones que comienza con el símbolo inicial S y termina con la cadena w . El lenguaje generado por G es el conjunto de todas las cadenas $w \in T^*$ que puede generar G .

Definición 1.9. *Sea G una gramática, el lenguaje generado por G es el conjunto:*

$$L(G) = \{x \in T^* \mid S \Rightarrow^+ x\}.$$

La jerarquía de Chomsky [7] clasifica las gramáticas en diferentes tipos según la forma de sus reglas de producción. Cada tipo de gramática es un subconjunto estricto del tipo anterior:

- Las gramáticas tipo-0, también conocidas como gramáticas irrestrictas, son las más generales. Sus reglas no tienen restricciones (son de la forma $\alpha \rightarrow \beta$ donde $\alpha \in (N \cup T)^* - T^*$ y $\beta \in (N \cup T)^*$). El conjunto de lenguajes que generan se denomina lenguajes recursivamente enumerables (RE).
- En las gramáticas de tipo-1, o gramáticas dependientes del contexto (CS), las reglas de producción son de la forma $\alpha A \gamma \rightarrow \alpha \beta \gamma$, donde $A \in N$, $\alpha, \beta, \gamma \in (N \cup T)^*$, y $|\beta| \geq 1$. Estas generan los lenguajes dependientes del contexto (CS).
- Las gramáticas de tipo-2, o gramáticas libres de contexto (CFG), son muy utilizadas en la construcción de compiladores [6]. Sus reglas de producción son de la forma $A \rightarrow \beta$, donde $A \in N$ y $\beta \in (N \cup T)^*$. Estas generan los lenguajes libres de contexto (CFL por sus siglas en inglés).
- Finalmente, las gramáticas de tipo-3, o gramáticas regulares, son gramáticas con reglas de producción de la forma $A \rightarrow aB$ o $A \rightarrow a$, donde $A, B \in N$ y $a \in T$. Estas gramáticas generan los lenguajes regulares (REG).

Definición 1.10. Sea $X \in \{RE, CS, CFG, REG\}$ una clase de gramáticas, la familia de lenguajes generados por X se define como:

$$\mathcal{L}(X) = \{L(G) \mid G \text{ es una gramática de tipo } X\}.$$

La jerarquía de Chomsky establece que:

$$\mathcal{L}(REG) \subset \mathcal{L}(CFG) \subset \mathcal{L}(CS) \subset \mathcal{L}(RE).$$

La diferencia entre los lenguajes de la jerarquía de Chomsky se puede ilustrar con el lenguaje $L_{0,1,d}$ definido como:

$$L_{0,1,d} = \{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\}.$$

Algunos ejemplos de cadenas que pertenecen a $L_{0,1,d}$ son $0d$, $1d$, $00d$, $10d$, $110d$, $0d0d$, $1d1d$, $00d00d00d00d$, $10d10d$ y $110d110d$.

Si se toma un caso particular de $L_{0,1,d}$, al cual se le llama

$$L_{0,1,d}^n = \{(wd)^n \mid w \in \{0,1\}^*\}$$

se cumple que $L_{0,1,d}^1$ es un lenguaje regular, mientras que $L_{0,1,d}^k$ para $k > 1$ es un lenguaje dependiente del contexto [6].

Los lenguajes regulares están relacionados con los autómatas finitos, los cuales se junto con el lenguaje dependiente del contexto $L_{0,1,d}$ se utilizan en el capítulo 2 para construir el lenguaje de las interpretaciones que satisfacen una fórmula booleana dada. En la siguiente sección se definen los autómatas finitos.

1.1.3. Lenguajes regulares y autómatas

Un autómata es un modelo matemático para describir sistemas que procesan cadenas. En la teoría de lenguajes formales, los autómatas se utilizan principalmente para determinar si una cadena pertenece a un lenguaje específico. Cada gramática de la jerarquía de Chomsky tiene un tipo de autómata asociado que reconoce exactamente los lenguajes generados por esa gramática [6]. A continuación se introduce el autómata finito determinista, que es el modelo de autómata asociado a los lenguajes regulares [6].

Definición 1.11. *Un autómata finito determinista (DFA por sus siglas en inglés) [6] es una 5-tupla:*

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q es un conjunto finito de estados.
- Σ es un alfabeto de entrada.
- $\delta : Q \times \Sigma \rightarrow Q$ es una función de transición que define el cambio de estado para cada símbolo de entrada.
- $q_0 \in Q$ es el estado inicial.
- $F \subseteq Q$ es el conjunto de estados de aceptación o finales.

Para describir cómo un DFA procesa una cadena de entrada, se define la función de transición extendida, la cual es una generalización de la función de transición δ para cadenas en lugar de símbolos individuales.

Definición 1.12. *La función de transición extendida $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ se define inductivamente como sigue:*

Caso base: Para cualquier estado $q \in Q$, $\hat{\delta}(q, \varepsilon) = q$.

Paso inductivo: Para cualquier estado $q \in Q$, símbolo $a \in \Sigma$ y cadena $w \in \Sigma^*$, se define:

$$\hat{\delta}(q, aw) = \hat{\delta}(\delta(q, a), w).$$

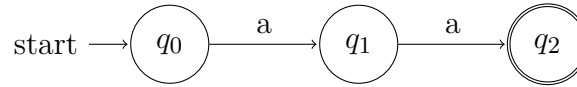


Figura 1.1: Autómata finito determinista que reconoce el lenguaje $\{aa\}$.

El autómata finito determinista acepta una cadena w si al aplicar la función de transición extendida al estado inicial q_0 y a la cadena w , se llega a un estado de aceptación. El lenguaje reconocido por un DFA es el conjunto de todas las cadenas que el autómata acepta.

Definición 1.13. *El lenguaje reconocido por un DFA $\mathcal{A}$ se define como:*

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}.$$

En la figura 1.1 se muestra un DFA que reconoce el lenguaje $\{aa\}$ sobre el alfabeto $\{a\}$. En la representación gráfica de los autómatas, La función de transición se representa con flechas. En este caso están representadas $\delta(q_0, a) = q_1$ y $\delta(q_1, a) = q_2$. El estado q_2 es el único estado de aceptación, por lo que el autómata acepta únicamente la cadena aa .

En [6] se demuestra que los lenguajes reconocidos por DFAs son exactamente los lenguajes regulares.

En el contexto de la teoría de lenguajes formales, resulta necesario analizar el costo computacional de las operaciones y problemas asociados a lenguajes. Para medir este costo se utiliza la teoría de la complejidad computacional. A continuación se introducen nociones básicas de complejidad computacional que serán útiles para dicho análisis.

1.2. Complejidad computacional

La teoría de la complejidad computacional clasifica problemas en función de los recursos necesarios para resolverlos. A continuación se introducen algunos conceptos de esta teoría, como la notación asintótica y las clases de problemas.

1.2.1. Notación asintótica

La notación asintótica se usa para describir el comportamiento de una función $f(n)$, donde n es el tamaño de la entrada. A continuación se define la notación O (grande):

Definición 1.14. *Sea $f(n)$ y $g(n)$ dos funciones de n . Se dice que $f(n) = O(g(n))$ si existen constantes positivas c y n_0 tales que:*

$$f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0.$$

Esta notación se utiliza para describir el límite superior del crecimiento de una función, lo cual se usa para analizar la eficiencia de algoritmos y la complejidad de problemas computacionales.

Se dice que un algoritmo requiere una cantidad de tiempo polinomial si puede ejecutarse en una complejidad de $O(n^k)$ donde n es el tamaño de la entrada del algoritmo y k es una constante. La complejidad de un problema es la complejidad del algoritmo más eficiente que lo resuelve en el peor caso.

En dependencia de los recursos computacionales necesarios para resolverlos, los problemas se agrupan en diferentes clases de complejidad. A continuación se presentan algunas de estas, las cuales clasifican problemas de decisión.

1.2.2. Clases de problemas

Los problemas de decisión se agrupan en diferentes clases según los recursos computacionales necesarios para resolverlos. A continuación se definen las clases de complejidad que se utilizarán a lo largo de este trabajo:

Definición 1.15. *(Clase P) La clase de complejidad P es el conjunto de problemas de decisión que pueden resolverse en tiempo polinomial [6].*

Definición 1.16. *(Clase NP) La clase de complejidad NP es el conjunto de problemas de decisión para los cuales una solución propuesta puede ser verificada en tiempo polinomial [6].*

Definición 1.17. *(Clase NP-completo) Un problema de decisión es NP-completo si pertenece a NP y además es tan difícil como cualquier otro problema en NP. Esto significa que cualquier problema en NP puede ser reducido a él en tiempo polinomial [6].*

Definición 1.18. *(Clase NP-duro) Un problema de decisión es NP-duro si es tan difícil como cualquier otro problema en NP pero no necesariamente pertenece a NP [6].*

Determinar si P es igual a NP es uno de los problemas abiertos más famosos en la ciencia de la computación. P es un subconjunto de NP, pero hasta la fecha no se sabe si esta inclusión es estricta o si ambos conjuntos son iguales [6].

El conjunto de los problemas NP-completos tiene la propiedad de que si se encuentra un algoritmo de tiempo polinomial para resolver cualquier problema NP-completo, entonces se demostraría que P es igual a NP. Por otro lado, si se demuestra que no existe tal algoritmo, entonces se confirmaría que P es diferente de NP [6].

En este trabajo se estudia el problema de la satisfacibilidad booleana, que es un problema de decisión NP-completo. A continuación se define este problema.

1.3. El problema de la satisfacibilidad booleana

El problema de la satisfacibilidad booleana (SAT) es un problema de decisión perteneciente a la clase NP-completo. En esta sección se define este problema y se analiza su importancia para la teoría de la complejidad computacional.

1.3.1. Definiciones

A continuación se definen los conceptos necesarios para formalizar el problema de satisfacibilidad booleana.

Definición 1.19. (*Variable booleana*) Una variable booleana es una variable que puede tomar solo dos valores: 1 (verdadero) o 0 (falso).

Definición 1.20. (*Literal*) Sea x una variable booleana, un literal o término es una variable x o su negación $\neg x$.

Definición 1.21. (*Cláusula*) Una cláusula C es una disyunción finita $t_1 \vee t_2 \vee \cdots \vee t_k$ de literales $t_1, \dots, t_k$.

Definición 1.22. (*Fórmula en forma normal conjuntiva*) Una fórmula booleana está en forma normal conjuntiva (CNF) si es una conjunción de cláusulas. Es decir, una fórmula F está en CNF si se puede escribir como:

$$F = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

donde cada C_i es una cláusula.

Definición 1.23. (*Interpretación*) Una interpretación es una asignación de valores de verdad a las variables de una fórmula booleana. Formalmente, sea F una fórmula y sea $X = \{x_1, \dots, x_n\}$ el conjunto de variables de F . Una interpretación es una función $v : X \rightarrow \{0, 1\}$.

Definición 1.24. (*Evaluación de una fórmula*) Sea v una interpretación. La evaluación de una fórmula booleana F bajo v , denotada por $(F)_v$, es el valor de verdad obtenido al sustituir cada variable x de F por el valor $v(x)$ y evaluar la fórmula según las reglas de la lógica booleana. Se dice que una interpretación v satisface una fórmula F si $(F)_v = 1$.

Definición 1.25. (*Problema de satisfacibilidad booleana*) Sea F una fórmula booleana, el problema de satisfacibilidad booleana (SAT) consiste en determinar si existe una interpretación v para las variables de F tal que $(F)_v = 1$.

En [6] se demuestra que para toda fórmula booleana existe una fórmula booleana en CNF que es satisfacible si y solo si la fórmula original es satisfacible. Por lo tanto, sin pérdida de generalidad, en lo que sigue se considerarán únicamente fórmulas booleanas en CNF. Por simplicidad, se las denominará simplemente fórmulas booleanas o fórmulas.

1.3.2. SAT como problema NP-completo

El problema de satisfacibilidad booleana es un problema de gran relevancia en la teoría de la complejidad computacional [6]. En particular, fue el primer problema demostrado como NP-completo por Stephen Cook en 1971 [5], lo que lo convierte en un punto de referencia para el estudio de la NP-completitud.

En consecuencia, SAT actúa como un problema representativo de la clase NP: resolverlo eficientemente implicaría la existencia de algoritmos eficientes para todos los problemas en NP.

1.3.3. SAT desde la perspectiva de lenguajes formales

El problema SAT puede ser abordado desde la perspectiva de lenguajes formales. En la Facultad de Matemática y Computación de la Universidad de La Habana se han realizado varios trabajos [1, 9, 11, 10]. Estos atacan el problema con teoría de lenguajes formales y buscan resolver instancias específicas en tiempo polinomial o demostrar que ciertas clases de lenguajes tienen problemas de decisión NP-duros mediante reducciones polinomiales de SAT.

En estos trabajos se toman dos enfoques principales utilizando el problema del vacío o de la palabra en lenguajes respectivamente. En el primer caso, se construye el lenguaje de las interpretaciones verdaderas de una fórmula y se determina si este es vacío o no, en caso de no ser vacío implica que existe una interpretación verdadera y que la fórmula es satisfacible. En el segundo caso se construye el lenguaje de todas las fórmulas satisfacibles y se determina si la fórmula dada pertenece a ese lenguaje.

En [1] primero se asume que todas las variables de la fórmula son distintas y se construye el lenguaje de las interpretaciones verdaderas de la fórmula bajo esa asunción. Se demuestra que dicho lenguaje es regular mediante la construcción de un autómata finito, el cual se denominó autómata booleano, que representa las reglas de la lógica proposicional en sus transiciones y sus estados representan un valor de verdad para la fórmula que se tiene hasta el momento. Por último se intersecta dicho autómata con un lenguaje libre del contexto que garantice que todas las instancias de la misma variable tengan el mismo valor de verdad, y se determina si el lenguaje resultante es vacío o no.

En [9] y [11] se generaliza la idea anterior y se exploran otros formalismos generadores de lenguajes y se analizan soluciones dadas de intersectar el autómata booleano con estos. En [11] además se introduce la idea de utilizar una variante del lenguaje *Copy* para garantizar que todas las instancias de la misma variable tengan el mismo valor de verdad.

En [10] se resuelve SAT mediante el enfoque del problema de la palabra. Se construye el lenguaje de las fórmulas satisfacibles mediante una transducción de una variante del lenguaje *Copy*, y además se propone un formalismo que reconoce este lenguaje. Como resultado se demuestra que el problema de la palabra para todos los formalismos que generen la variante del lenguaje *Copy* y sean cerrados bajo transducción finita es NP-duro.

En el siguiente capítulo se propone un método para resolver SAT mediante el enfoque del problema del vacío, el cual no depende del orden de las variables de la fórmula. Este se basa en la construcción del lenguaje de interpretaciones que satisfacen una fórmula y determinando si este es vacío o no.

Capítulo 2

SAT y el problema del vacío para la intersección con un lenguaje regular

En esta sección se presenta el lenguaje $L_{\text{sat}}(F)$ al cual pertenecen todas las interpretaciones que satisfacen a una fórmula F y se muestra una forma de construirlo a partir de la intersección del lenguaje $L_{0,1,d}$ con un autómata finito determinista. Esta construcción permite resolver el problema de la satisfacibilidad booleana (SAT) para una fórmula F a través de la decisión de si el lenguaje $L_{\text{sat}}(F)$ es vacío o no.

El capítulo se estructura de la siguiente forma: en la sección 2.1 se muestra cómo codificar las interpretaciones de una fórmula usando el alfabeto $\{0, 1, d\}$ y se define el lenguaje $L_{\text{sat}}(F)$. En la sección 2.2 se relaja el problema de satisfacibilidad booleana para permitir que cada cláusula de la fórmula se satisfaga por una interpretación distinta, y se construye un autómata que reconoce el lenguaje de las cadenas que representan a las interpretaciones que satisfacen a cada cláusula de la fórmula. Finalmente, en la sección 2.3 se muestra cómo obtener el lenguaje $L_{\text{sat}}(F)$ a partir de la intersección del lenguaje reconocido por dicho autómata con el lenguaje $L_{0,1,d}$. Por último, se presenta por qué para todo formalismo que represente el lenguaje $L_{0,1,d}$ decidir si es vacío o construir la intersección con un lenguaje regular son problemas NP-duros.

A continuación se presenta cómo codificar una interpretación de una fórmula booleana como una cadena sobre el alfabeto $\{0, 1, d\}$.

2.1. Codificación de las interpretaciones de una fórmula booleana como cadenas

Una formula booleana F , con n variables $X = \{x_1, x_2, \dots, x_n\}$ y m cláusulas tiene la siguiente estructura:

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

donde cada cláusula C_i es una disyunción de literales

$$C_i = t_{i1} \vee t_{i2} \vee \dots \vee t_{ik_i}$$

y cada literal t_{ij} es una variable x_l o su negación $\neg x_l$ para algún $l \in \{1, \dots, n\}$.

El problema de satisfacibilidad booleana (SAT) consiste en decidir si existe una asignación de valores a las variables que haga verdadera a la fórmula.

Para representar las interpretaciones de F como cadenas, se asignará a cada variable un símbolo binario: 0 para falso y 1 para verdadero. De esta forma, cada interpretación v de las variables de F se puede representar como una cadena de longitud n sobre el alfabeto $\{0, 1\}$, donde el i -ésimo símbolo de la cadena representa el valor asignado a la variable x_i por la interpretación v .

Por ejemplo, si $X = \{x_1, x_2, x_3\}$ y una interpretación v asigna $v(x_1) = 0$, $v(x_2) = 1$ y $v(x_3) = 0$, entonces la cadena que representa a esta interpretación sería 010.

En este trabajo se propone resolver el problema de la satisfacibilidad booleana reduciéndolo a determinar si un lenguaje es vacío o no.

Para ello se codificarán las interpretaciones de la siguiente manera:

1. A cada interpretación v se le asigna una cadena w de longitud n sobre el alfabeto $\{0, 1\}$, donde el i -ésimo símbolo de la cadena representa el valor asignado a la variable x_i por la interpretación.
2. Se introduce un nuevo símbolo d que se utilizará como separador entre las cláusulas de la fórmula.
3. Se copia la cadena w asociada a cada interpretación v m veces, una por cada cláusula de la fórmula, y se separan con el símbolo d .

De esta forma, a cada interpretación v de las variables de F se le asocia una cadena de la forma $(wd)^m$, donde w es la cadena binaria que representa a la interpretación v y d es el símbolo separador.

Por ejemplo, para la fórmula $F = (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2)$, si una interpretación v asigna $v(x_1) = 0$, $v(x_2) = 1$ y $v(x_3) = 0$, entonces la cadena que se le asociará a esta interpretación será 010d010d.

En el resto del capítulo, dada una interpretación v de una fórmula con n variables y m cláusulas, se dirá que la cadena $w \in \{0, 1\}^n$ codifica a v , o que representa la

interpretación v , y se denominará representación extendida de v a la cadena $(wd)^m$ asociada.

El lenguaje de todas las interpretaciones que satisfacen a una fórmula F de m cláusulas se puede definir como el siguiente lenguaje sobre el alfabeto $\{0, 1, d\}$:

$$L_{\text{sat}}(F) = \{(wd)^m \mid w \text{ codifica a una interpretación que satisface } F = C_1 \wedge \dots \wedge C_m\}.$$

Como para cada interpretación v existe una y solo una cadena asociada en $L_{\text{sat}}(F)$, entonces el lenguaje $L_{\text{sat}}(F)$ es vacío si y solo si no existe una interpretación que haga verdadera a F .

Para decidir si existe una interpretación que haga verdadera a F basta con decidir si el lenguaje $L_{\text{sat}}(F)$ es vacío o no. Por tanto, se puede reducir el problema SAT para una fórmula F a decidir si el lenguaje $L_{\text{sat}}(F)$ es vacío o no.

En el resto del capítulo se describe como construir el lenguaje $L_{\text{sat}}(F)$. En la siguiente sección se presenta un autómata que reconoce el lenguaje de las cadenas que permiten que cada cláusula se satisfaga por una interpretación distinta. En la sección 2.3 se muestra como obtener el lenguaje $L_{\text{sat}}(F)$ a partir de la intersección del lenguaje reconocido por dicho autómata con el lenguaje $L_{0,1,d}$.

2.2. Autómata para el lenguaje $L_{\text{ind}}(F)$

En esta sección se considera una versión relajada del problema, en la que se permite que cada cláusula de la fórmula se satisfaga por una interpretación distinta, en lugar de exigir una única interpretación que satisfaga a todas las cláusulas simultáneamente. Bajo esta relajación se define el lenguaje

$$L_{\text{ind}}(F) = \{w_1 d \dots w_m d \mid w_i \text{ codifica una interpretación que satisface la cláusula } C_i\}$$

y se construye un autómata $\mathcal{A}_F$ que lo reconoce. Más adelante, en la sección 2.3, al intersectar este lenguaje con $L_{0,1,d}$ se recuperará la restricción de que todas las cláusulas compartan la misma interpretación.

Para construir dicho autómata se construirá primero un autómata $\mathcal{A}_C$ para cada cláusula de la fórmula, y luego estos se concatenarán para obtener el autómata que reconoce el lenguaje $L_{\text{ind}}(F)$. A continuación se describe la construcción del autómata para cada cláusula de la fórmula.

2.2.1. Automata cláusula

En esta sección se construye el autómata $\mathcal{A}_C$ que reconoce el lenguaje de las cadenas que representan a las interpretaciones que satisfacen a una cláusula C de

una fórmula n variables $X = \{x_1, \dots, x_n\}$. Se denotará a este lenguaje L_C y se define como sigue:

$$L_C = \{w \mid w \text{ representa a una interpretación que satisface la cláusula } C\}$$

Para una fórmula con n variables, los estados del autómata $\mathcal{A}_C$ son de la forma (q_f, i) o (q_t, i) , donde $i \in \{0, \dots, n\}$. Estos estados representan la información de si luego de leer los i primeros símbolos de la cadena, ya se encontró un literal de la cláusula que evalúa verdadero o no.

- Un estado de la forma (q_f, i) representan que luego de leer i símbolos de la cadena no se ha leído el valor de una variable x_j con $1 \leq j \leq i$ cuyo literal asociado en la cláusula sea verdadero.
- Un estado de la forma (q_t, i) representan que luego de leer i símbolos de la cadena, ya se ha leído el valor de una variable x_j con $1 \leq j \leq i$ cuyo literal asociado en la cláusula es verdadero.

Por ejemplo, un estado $(q_f, 2)$ representa que luego de leer los valores de las dos primeras variables de la interpretación, no se ha leído el valor de ninguna variable cuyo literal asociado en la cláusula evalúe verdadero bajo dicha interpretación. En cambio un estado $(q_t, 7)$ significa que luego de leer los valores de las primeras 7 variables de la interpretación, se ha leído el valor de una variable cuyo literal asociado en la cláusula evalúa verdadero bajo dicha interpretación.

El estado inicial del autómata será $(q_f, 0)$, pues al no haber leído ningún símbolo de la cadena, no se ha leído el valor asociado a ninguna variable y por tanto la cláusula no puede ser verdadera.

El estado de aceptación del autómata será (q_t, n) , y significa que luego de leer los n símbolos de la cadena, se ha leído el valor de una variable cuyo literal asociado en la cláusula es verdadero, por lo tanto la cláusula es verdadera.

Las transiciones entre los estados solo existen entre estados de la forma (q_f, i) o (q_t, i) y estados de la forma $(q_f, i+1)$ o $(q_t, i+1)$.

- Un estado de la forma (q_t, i) representa que luego de leer i símbolos de la cadena, se ha leído el valor de una variable cuyo literal asociado en la cláusula es verdadero, por lo tanto, sin importar los símbolos restantes, la cláusula seguirá siendo verdadera. Luego, se tiene la transición

$$\delta((q_t, i), a) = (q_t, i+1) \text{ para cualquier símbolo } a \in \{0, 1\}.$$

- Por otro lado un estado de la forma (q_f, i) representa que luego de leer i símbolos de la cadena, no se ha leído el valor de una variable cuyo literal asociado en

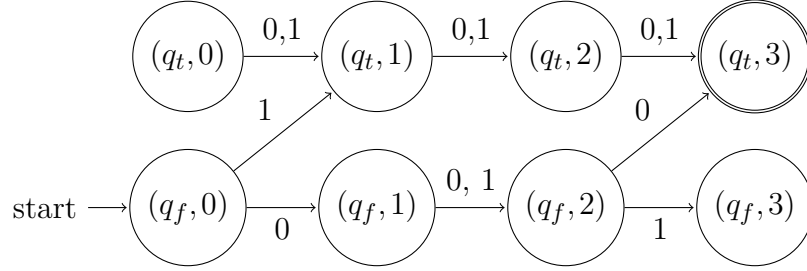


Figure 2.1: Autómata cláusula asociado a la cláusula $C = (x_1 \vee \neg x_2)$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$.

la cláusula sea verdadero, por lo tanto, para que la cláusula sea verdadera, el siguiente símbolo leído debe ser tal que el literal asociado a la variable x_{i+1} en la cláusula evalúe verdadero. Si la variable x_{i+1} no aparece en la cláusula, o aparece pero el símbolo leído hace que el literal asociado evalúe falso, entonces la cláusula seguirá siendo falsa. Por lo tanto se tiene la transición

$$\delta((q_f, i), a) = \begin{cases} (q_t, i+1) & \text{si } (x_{i+1} \in C \wedge a = 1) \vee (\neg x_{i+1} \in C \wedge a = 0) \\ (q_f, i+1) & \text{en otro caso} \end{cases}$$

En la figura 2.1 se muestra el autómata cláusula asociado a la cláusula $C = x_1 \vee \neg x_2$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$.

A continuación se define formalmente el autómata cláusula.

Definición 2.1. Para una cláusula C de una fórmula F con n variables, se define el autómata cláusula $\mathcal{A}_C = (Q, \{0, 1\}, \delta, (q_f, 0), F)$ donde:

- $Q = \{(q_f, i), (q_t, i) \mid i \in \{0, \dots, n\}\}$
- $(q_f, 0)$ es el estado inicial
- δ es la función de transición definida para todo $i \in \{0, \dots, n-1\}$ y $a \in \{0, 1\}$ por:

$$\begin{aligned} \delta((q_t, i), a) &= (q_t, i+1), \\ \delta((q_f, i), a) &= \begin{cases} (q_t, i+1) & \text{si } (x_{i+1} \in C \wedge a = 1) \vee (\neg x_{i+1} \in C \wedge a = 0), \\ (q_f, i+1) & \text{en otro caso.} \end{cases} \end{aligned}$$

- $F = \{(q_t, n)\}$

Para una fórmula con n variables, el autómata $\mathcal{A}_C$ tendrá $2(n+1) = O(n)$ estados: $n+1$ de la forma (q_f, i) y $n+1$ de la forma (q_t, i) , con $i \in \{0, \dots, n\}$.

A continuación se demostrará que el lenguaje que reconoce el autómata $\mathcal{A}_C$ es exactamente el lenguaje

$$L_C = \{w \mid w \text{ codifica a una interpretación que satisface la cláusula } C\}.$$

Para ello, primero se demostrará el siguiente lema que describe la propiedad mantenida por el autómata luego de leer k variables.

Lema 2.1. *Dado un autómata cláusula $\mathcal{A}_C$ y una cadena de entrada $w = w_1 w_2 \dots w_r$, luego de leer $k \leq r$ símbolos el autómata $\mathcal{A}_C$ se encuentra en un estado verdadero (q_t, k) si y solo si existe una variable x_i de C con $1 \leq i \leq k$ tal que $w_i = 1 \wedge x_i \in C$ o $w_i = 0 \wedge \neg x_i \in C$.*

Demostración Primero se demostrará que si luego de leer k símbolos el autómata $\mathcal{A}_C$ se encuentra en un estado verdadero entonces existe una variable x_i de C tal que el literal asociado en C es verdadero. Formalmente:

$$\hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_t, k) \implies \exists i \in \{1, \dots, k\} \text{ con } (a_i = 1 \wedge x_i \in C) \vee (a_i = 0 \wedge \neg x_i \in C). \quad (2.1)$$

Se demostrará por inducción sobre k .

Caso base Para $k = 1$ si se tiene que $\delta((q_f, 0), a_1) = (q_t, 1)$, entonces por la definición de la función de transición se tiene que a_1 es tal que el literal asociado a la variable x_1 en C es verdadero. Por lo tanto, se cumple la afirmación.

Paso inductivo Supóngase que la afirmación se cumple para k . Supóngase además que luego de leer $k+1$ símbolos el autómata se encuentra en un estado verdadero, formalmente esto es $\hat{\delta}((q_f, 0), a_1 \dots a_{k+1}) = (q_t, k+1)$. Se tienen dos casos:

- $\hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_t, k)$, entonces por la hipótesis inductiva existe una variable x_i de C con $i \leq k$ tal que el literal asociado en C es verdadero, por lo tanto se cumple la afirmación.
- $\hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_f, k)$, entonces por la definición de la función de transición se tiene que a_{k+1} es tal que el literal asociado a la variable x_{k+1} en C es verdadero. Por lo tanto, se cumple la afirmación.

Por tanto para todo $k \in \{1, \dots, n\}$ se cumple que si el autómata luego de leer k símbolos se encuentra en un estado verdadero, entonces existe una variable x_i de C con $i \leq k$ tal que el literal asociado en C es verdadero.

A continuación se demuestra la implicación recíproca: si existe una variable x_i de C con $i \leq k$ tal que el literal asociado en C es verdadero, entonces el autómata luego de leer k símbolos se encuentra en un estado verdadero. Formalmente:

$$\exists i \in \{1, \dots, k\} \text{ con } (a_i = 1 \wedge x_i \in C) \vee (a_i = 0 \wedge \neg x_i \in C) \implies \hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_t, k). \quad (2.2)$$

Se demostrará por inducción sobre k .

Caso base Para $k = 1$ se cumple que $x_1 \in C$ y $a_1 = 1$ o $\neg x_1 \in C$ y $a_1 = 0$, entonces por la definición de la función de transición se tiene que $\delta((q_f, 0), a_1) = (q_t, 1)$, por lo tanto $\hat{\delta}((q_f, 0), a_1) = (q_t, 1)$ y se cumple la afirmación.

Paso inductivo Supóngase que la afirmación se cumple para k : existe una variable x_i de C con $i \leq k + 1$ tal que el literal asociado en C es verdadero.

Se tienen dos casos: existe un literal x_j con $j \leq k$ tal que el literal asociado a la variable x_j en C es verdadero, o solo el literal asociado a la variable x_{k+1} en C es verdadero.

- Primer caso. Existe $i \leq k$ tal que el literal asociado a la variable x_i en C es verdadero, entonces por hipótesis inductiva,

$$\hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_t, k).$$

Como $\hat{\delta}((q_t, k), a_{k+1}) = (q_t, k + 1)$, para todo $a_{k+1} \in \{0, 1\}$ concluye que

$$\hat{\delta}((q_f, 0), a_1 \dots a_{k+1}) = (q_t, k + 1),$$

y por lo tanto se cumple la afirmación.

- En el segundo caso, solo el literal asociado a x_{k+1} en C es verdadero. Esto significa que:

$$(a_{k+1} = 1 \wedge x_{k+1} \in C) \vee (a_{k+1} = 0 \wedge \neg x_{k+1} \in C).$$

Entonces, por definición de δ se tiene que

$$\delta((q_f, k), a_{k+1}) = (q_t, k + 1)$$

Como para todo $i \leq k$ el literal asociado a x_i en C es falso, por la implicación 2.1 ya demostrada, se tiene que

$$\hat{\delta}((q_f, 0), a_1 \dots a_k) \neq (q_t, k).$$

Dado que luego de leer k símbolos el autómata no se encuentra en un estado verdadero, se tiene que

$$\hat{\delta}((q_f, 0), a_1 \dots a_k) = (q_f, k).$$

Por tanto,

$$\hat{\delta}((q_f, 0), a_1 \dots a_{k+1}) = (q_t, k+1).$$

Con esto se cumple la afirmación.

Luego de demostrar ambas implicaciones se concluye que para todo $k \in \{1, \dots, n\}$ se cumple que luego de leer k símbolos el autómata se encuentra en un estado verdadero si y solo si existe una variable x_i de C con $i \leq k$ tal que el literal asociado en C es verdadero. $\square$

Ahora se procederá a enunciar y demostrar el teorema que establece la equivalencia entre el lenguaje reconocido por el autómata $\mathcal{A}_C$ y el lenguaje L_C .

Teorema 2.1. *Sean C una cláusula de una fórmula con variables $X = \{x_1, \dots, x_n\}$, $\mathcal{A}_C$ el autómata cláusula asociado a C y L_C el lenguaje de las cadenas que representan a las interpretaciones que satisfacen a C . Se cumple que $L_C = L(\mathcal{A}_C)$.*

Demostración Para demostrar que $L_C = L(\mathcal{A}_C)$ se deben demostrar dos inclusiones: primero se demostrará que $L_C \subseteq L(\mathcal{A}_C)$ y luego que $L(\mathcal{A}_C) \subseteq L_C$.

$L_C \subseteq L(\mathcal{A}_C)$:

Supóngase que $w \in L_C$, entonces $w = a_1 \dots a_n$ representa una interpretación v tal que $(C)_v = 1$, por tanto existe un literal t de C tal que $(t)_v = 1$. Por el lema 2.1 con $k = n$ se cumple que $\hat{\delta}((q_f, 0), w) = (q_t, n)$, luego $w \in L(\mathcal{A}_C)$.

$L(\mathcal{A}_C) \subseteq L_C$:

Supóngase que $w \in L(\mathcal{A}_C)$, entonces $\hat{\delta}((q_f, 0), w) = (q_t, n)$, por el lema 2.1 con $k = n$ se cumple que existe un literal t de C tal que $(t)_v = 1$, luego $(C)_v = 1$ y w representa a una interpretación v tal que $(C)_v = 1$, por tanto $w \in L_C$.

Como se tiene que $L_C \subseteq L(\mathcal{A}_C)$ y $L(\mathcal{A}_C) \subseteq L_C$, entonces

$$L_C = L(\mathcal{A}_C).$$

$\square$

A continuación se describirá la construcción del autómata que reconoce el lenguaje L_F a partir de los autómatas $\mathcal{A}_C$ asociados a cada cláusula de la fórmula F .

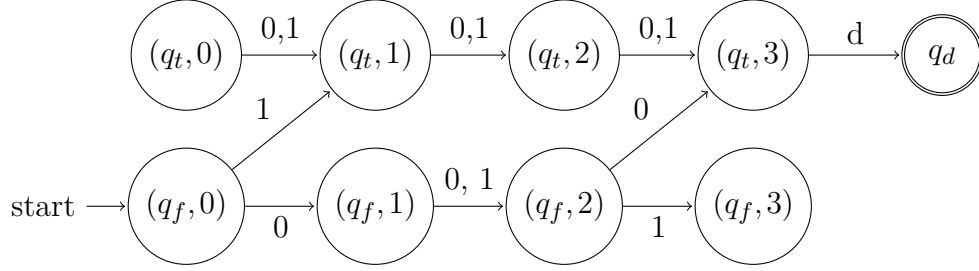


Figure 2.2: Autómata cláusula con separador asociado a la cláusula $C = (x_1 \vee \neg x_2)$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$.

2.2.2. Autómata fórmula

Dada la fórmula F con cláusulas $C_1, \dots, C_m$, se tienen autómatas $\mathcal{A}_{C_i}$ que reconocen el lenguaje L_{C_i} para cada cláusula C_i de la fórmula, bajo la suposición en la que se permite que cada cláusula se satisfaga por una interpretación distinta. Para construir el autómata que reconoce el lenguaje

$$L_{\text{ind}}(F) = \{w_1 d \dots w_m d \mid w_i \text{ codifica una interpretación que satisface la cláusula } C_i\}.$$

se necesita concatenar los autómatas $\mathcal{A}_{C_i}$ para cada cláusula C_i de la fórmula, pero antes es necesario modificar estos autómatas para que reconozcan las cadenas que representan a una interpretación que satisface a la cláusula con el símbolo separador d al final. De esta forma, para cada cláusula C_i el autómata resultante reconocerá las cadenas de la forma $w_i d$, donde w_i representa a una interpretación que satisface a la cláusula C_i .

Primero se construye un autómata que reconozca el lenguaje L_C con los separadores de cláusulas: el lenguaje $L_{Cd} = \{(wd) \mid w \in L_C\}$. Para esto basta con modificar el autómata $\mathcal{A}_C$ para que si está en el estado de aceptación reconozca la cadena d después de haber leído una cadena de L_C .

A continuación se define formalmente el autómata cláusula con separador asociado a una cláusula C_i de la fórmula F .

Definición 2.2. Dada una cláusula C_i de una fórmula con las variables $X = \{x_1, \dots, x_n\}$ sea

$$\mathcal{A}_{C_i} = (Q, \{0, 1\}, \delta, (q_f, 0), \{(q_t, n)\})$$

el autómata cláusula asociado a C_i . Se define el autómata cláusula con separador asociado a C_i como

$$\mathcal{A}_{C_i}^d = (Q \cup \{q_d\}, \{0, 1, d\}, \delta', (q_f, 0), \{q_d\})$$

donde δ' se define como

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{si } q \in Q \text{ y } q \neq (q_t, n) \\ q_d & \text{si } q = (q_t, n) \text{ y } a = d \end{cases}$$

El autómata $\mathcal{A}_{C_i}^d$ reconoce el lenguaje de las cadenas que representan a las interpretaciones que satisfacen a la cláusula C_i con el símbolo separador d al final. En la figura 2.2 de la página 23 se muestra el autómata cláusula con separador asociado a la cláusula $C = x_1 \vee \neg x_2$ de una fórmula con variables $X = \{x_1, x_2, x_3\}$.

Los autómatas $\mathcal{A}_{C_i}^d$ para cada cláusula C_i reconocen el lenguaje de las cadenas que representan a las interpretaciones que satisfacen a la cláusula C_i con el símbolo separador d al final. Formalmente se tiene el siguiente teorema.

Teorema 2.2. *Sea una cláusula C_i de una fórmula con variables $X = \{x_1, \dots, x_n\}$, $\mathcal{A}_{C_i}^d$ el autómata cláusula con separador asociado a C_i y el lenguaje L_{C_i} de las cadenas que representan a las interpretaciones que satisfacen a la cláusula C_i . Se cumple que*

$$L(\mathcal{A}_{C_i}^d) = \{(wd) \mid w \in L_{C_i}\}.$$

La demostración es directa dada la definición de δ' y el hecho de que $L_{C_i} = L(\mathcal{A}_{C_i})$. $\square$

El siguiente paso es concatenar los autómatas cláusula con separador para construir un autómata que reconozca el lenguaje $L_{\text{ind}}(F)$.

Para ello, se concatenan los autómatas $\mathcal{A}_{C_i}^d$ para cada cláusula C_i de la fórmula mediante el procedimiento de concatenación de [6]: se conecta el estado de aceptación q_{di} del autómata $\mathcal{A}_{C_i}^d$ con el estado inicial $(q_{f(i+1)}, 0)$ del autómata $\mathcal{A}_{C_{i+1}}^d$ para cada $i < m$, mediante la transición

$$\delta((q_{ti}, n), \varepsilon) = (q_{f(i+1)}, 0).$$

Estas transiciones introducen indeterminismo en el autómata. Sin embargo, existe una construcción estándar para convertir un autómata no determinista a un autómata determinista [6]. Esta construcción puede tomar un tiempo exponencial en el número de estados del autómata no determinista. No obstante, en este caso particular la construcción es inmediata, pues las transiciones ε solo conectan el estado final de un autómata con el estado inicial del siguiente. Esto permite que el indeterminismo pueda ser eliminado sustituyendo ambos estados por uno nuevo y modificando la función de transición.

A continuación se define formalmente el autómata fórmula asociado a una fórmula F con cláusulas $C_1, \dots, C_m$.

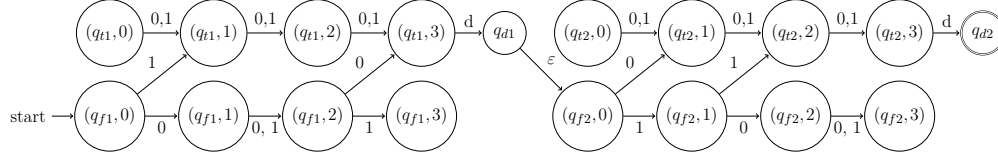


Figure 2.3: Autómata fórmula asociado a la fórmula $F = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$.

Definición 2.3. Sean una fórmula $F = C_1 \wedge \dots \wedge C_m$ con variables $X = \{x_1, \dots, x_n\}$. Sea

$$\mathcal{A}_{C_i}^d = (Q_i, \{0, 1, d\}, \delta_i, (q_{fi}, 0), (q_{di}))$$

el autómata cláusula con separador asociado a cada cláusula C_i de la fórmula F . Se define el autómata fórmula asociado a F como

$$\mathcal{A}_F = (Q, \{0, 1, d\}, \delta, (q_{f1}, 0), \{q_{dm}\})$$

donde:

- $Q = \bigcup_{i=1}^m Q_i$
- La función de transición δ es la unión de las funciones de transición δ_i y las transiciones $\delta((q_{di}, n), \varepsilon) = (q_{f(i+1)}, 0)$ para cada $i < m$.

Con la construcción del autómata fórmula se ha demostrado el siguiente teorema.

Teorema 2.3. El autómata $\mathcal{A}_F$ reconoce el lenguaje $L_{\text{ind}}(F)$ definido como

$$L_{\text{ind}}(F) = \{w_1 d \dots w_m d \mid w_i \text{ codifica una interpretación que satisface la cláusula } C_i\}.$$

Dado que los lenguajes regulares son aquellos que pueden ser reconocidos por un autómata finito [6], se tiene como resultado que el lenguaje $L_{\text{ind}}(F)$ es un lenguaje regular.

Colorario 2.1. El lenguaje $L_{\text{ind}}(F)$ es un lenguaje regular.

Las cadenas del lenguaje $L_{\text{ind}}(F)$ no necesariamente corresponden a la representación extendida de una interpretación que satisface a la fórmula F , ya que cada cláusula de la fórmula se puede satisfacer por una interpretación diferente.

Por ejemplo, dada la fórmula $F = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$, la cadena 101d001d pertenece a $L_{\text{ind}}(F)$ pero no representa una interpretación F . Esto se debe a que 101 y 001 son cadenas que representan dos interpretaciones diferentes que satisfacen a cada cláusula de la fórmula F .

En la siguiente sección se demostrará como obtener el lenguaje $L_{\text{sat}}(F)$ de todas las interpretaciones que satisfacen a la fórmula F a partir de intersectar el lenguaje $L_{\text{ind}}(F)$ con el lenguaje $L_{0,1,d}$ que obliga a que las interpretaciones sean las mismas para todas las cláusulas de la fórmula.

2.3. $L_{\text{sat}}(F)$ a partir de la intersección con un lenguaje regular

En esta sección se demostrará que el lenguaje $L_{\text{sat}}(F)$ de todas las interpretaciones que satisfacen a la fórmula F se puede obtener a partir de la intersección del lenguaje $L_{\text{ind}}(F)$ con el lenguaje $L_{0,1,d}$.

Dada una fórmula en forma normal conjuntiva

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

el lenguaje $L_{\text{ind}}(F)$ es el lenguaje de las cadenas $w_1dw_2d\dots dw_md$, donde w_i representa a una interpretación v_i que satisface a la cláusula C_i . Dado que es posible que existan $i, j \leq m$ tales que $w_i \neq w_j$, las cadenas de $L_{\text{ind}}(F)$ no necesariamente están asociadas a una interpretación que satisface la fórmula F .

Para obtener el lenguaje $L_{\text{sat}}(F)$ de todas las interpretaciones que satisfacen a la fórmula F , es necesario imponer la restricción de que las interpretaciones v_i asociadas a cada cláusula C_i sean iguales. Para ello se intersectará el lenguaje $L_{\text{ind}}(F)$ con el lenguaje $L_{0,1,d}$. Este lenguaje se define a continuación.

Definición 2.4. *El lenguaje $L_{0,1,d}$ se define como:*

$$L_{0,1,d} = \{(wd)^k \mid w \in \{0,1\}^*, k \geq 1\}$$

$L_{0,1,d}$ contiene cadenas sobre el alfabeto $\{0,1,d\}$ que representan una cadena binaria concatenada con el símbolo d repetida una o más veces.

El lenguaje $L_{0,1,d}$ obliga a que las cadenas que representan a las interpretaciones asociadas a cada cláusula sean iguales.

Por ejemplo, las cadenas $101d101d101d$, $101d101d$ y $101d$ pertenecen a $L_{0,1,d}$, mientras que las cadenas $101d100d$ y $101d100d101d$ no pertenecen a $L_{0,1,d}$.

A continuación se demuestra que el lenguaje $L_{\text{ind}}(F) \cap L_{0,1,d}$ es exactamente el lenguaje $L_{\text{sat}}(F)$ de todas las interpretaciones que hacen verdadera a F .

Teorema 2.4. *Para toda fórmula F con n variables y m cláusulas, sean:*

- $L_{\text{sat}}(F)$ es el lenguaje de todas las interpretaciones que satisfacen a la fórmula F ,
- $L_{\text{ind}}(F)$ es el lenguaje de las cadenas $w_1dw_2d\dots dw_md$, donde w_i representa a una interpretación v_i que satisface a la cláusula C_i , y
- $L_{0,1,d}$ es el lenguaje de las cadenas de la forma $(wd)^k$ con $w \in \{0,1\}^*$ y $k \geq 1$.

Entonces se cumple que $L_{\text{sat}}(F) = L_{\text{ind}}(F) \cap L_{0,1,d}$

Demostración Para demostrar que $L_{\text{sat}}(F) = L_{\text{ind}}(F) \cap L_{0,1,d}$ se demostrará primero que $L_{\text{sat}}(F) \subseteq L_{\text{ind}}(F) \cap L_{0,1,d}$ y luego que $L_{\text{ind}}(F) \cap L_{0,1,d} \subseteq L_{\text{sat}}(F)$.

$$L_{\text{sat}}(F) \subseteq L_{\text{ind}}(F) \cap L_{0,1,d}:$$

Sea $\alpha \in L_{\text{sat}}(F)$, por definición de $L_{\text{sat}}(F)$, se tiene que α es una cadena de la forma $(wd)^m$ donde w representa a una interpretación v que satisface a la fórmula F .

Se tiene que $\alpha \in L_{0,1,d}$, ya que α es una cadena de la forma $(wd)^m$ con $w \in \{0,1\}^*$ y $m \geq 1$.

Por otro lado, como v satisface a la fórmula F , entonces v satisface todas las cláusulas de F . Por lo tanto, $\alpha = w_1d \dots w_md$ donde $w_i = w$ representa a la interpretación v que satisface a la cláusula C_i para todo $i \leq m$, por lo tanto $\alpha \in L_{\text{ind}}(F)$.

Al α pertenecer a ambos conjuntos se concluye que $\alpha \in L_{\text{ind}}(F) \cap L_{0,1,d}$, luego $L_{\text{sat}}(F) \subseteq L_{\text{ind}}(F) \cap L_{0,1,d}$.

$$L_{\text{ind}}(F) \cap L_{0,1,d} \subseteq L_{\text{sat}}(F):$$

Sea $\alpha \in L_{\text{ind}}(F) \cap L_{0,1,d}$. Como $\alpha \in L_{0,1,d}$, entonces existen $w \in \{0,1\}^*$ y $k \geq 1$ tales que

$$\alpha = (wd)^k. \quad (2.3)$$

Como $\alpha \in L_{\text{ind}}(F)$, entonces

$$\alpha = w_1dw_2d \dots dw_md \quad (2.4)$$

donde w_i representa a una interpretación v_i que satisface a la cláusula C_i para todo $i \leq m$. Juntando las ecuaciones 2.3 y 2.4 se tiene

$$w_1dw_2d \dots dw_md = (wd)^k. \quad (2.5)$$

Como en el lado izquierdo de 2.4 el símbolo separador aparece exactamente m veces, entonces $k = m$. Luego,

$$w_1dw_2d \dots dw_md = (wd)^m. \quad (2.6)$$

Por tanto las cadenas separadas por el símbolo d son iguales a w , esto es,

$$w_1 = w_2 = \dots = w_m = w,$$

al ser w la cadena que representa a una interpretación v , entonces w_i representa a la misma interpretación v para todo $i \leq m$.

Por lo tanto, existe una interpretación v tal que $v_i = v$ para todo $i \leq m$ que satisface todas las cláusulas de la fórmula, y por tanto satisface la fórmula. Luego $\alpha = (wd)^m \in L_{\text{sat}}(F)$ con $(F)_v = 1$, por tanto $\alpha \in L_{\text{sat}}(F)$. Luego $L_{\text{ind}}(F) \cap L_{0,1,d} \subseteq L_{\text{sat}}(F)$.

Como se ha demostrado que $L_{\text{sat}}(F) \subseteq L_{\text{ind}}(F) \cap L_{0,1,d}$ y que $L_{\text{ind}}(F) \cap L_{0,1,d} \subseteq L_{\text{sat}}(F)$, se concluye que $L_{\text{sat}}(F) = L_{\text{ind}}(F) \cap L_{0,1,d}$. $\square$

Una consecuencia directa del teorema 2.4 es el siguiente resultado.

Teorema 2.5. *Para cualquier formalismo G que genere el lenguaje $L_{0,1,d}$ y sea cerrado bajo intersección con lenguajes regulares, la construcción de la intersección con lenguajes regulares o el problema de vacío es NP-duro.*

Demostración Supóngase que existe un formalismo G que genera el lenguaje $L_{0,1,d}$ y es cerrado bajo intersección con lenguajes regulares. Se demostrará que el problema de decidir si la intersección de un lenguaje generado por dicho formalismo con un lenguaje regular es vacía es NP-duro. Para ello, se reducirá el problema SAT a dicho problema.

Sea F una fórmula booleana con n variables y m cláusulas.

Se construye el autómata $\mathcal{A}_F$ que reconoce el lenguaje $L_{\text{ind}}(F)$. Luego se construye el lenguaje $L_{0,1,d}$ a partir del formalismo G . Dado que G es cerrado bajo intersección con lenguajes regulares, entonces se puede construir el lenguaje $L_{\text{ind}}(F) \cap L_{0,1,d}$ a partir de la intersección de $L_{\text{ind}}(F)$ y $L_{0,1,d}$.

Por el teorema 2.4 se tiene que $L_{\text{sat}}(F) = L_{\text{ind}}(F) \cap L_{0,1,d}$, por lo tanto decidir si existe una interpretación que hace verdadera a F es equivalente a decidir si el lenguaje $L_{\text{ind}}(F) \cap L_{0,1,d}$ es vacío o no.

La construcción del autómata $\mathcal{A}_F$ es polinomial en el tamaño de la fórmula F , ya que $\mathcal{A}_F$ tiene $O(n)$ estados, donde n es el número de variables de F . Por tanto, SAT se reduce al problema de obtener la intersección de un lenguaje regular con un lenguaje generado por el formalismo G y decidir si el resultado es vacío.

En consecuencia, al menos uno de estos problemas debe ser NP-duro:

- construir la intersección con un lenguaje regular, o
- decidir si el lenguaje resultante es vacío.

Luego, para cualquier formalismo que genere el lenguaje $L_{0,1,d}$ y sea cerrado bajo intersección con lenguajes regulares, la construcción de dicha intersección o el problema del vacío asociado son NP-duros. $\square$

Una restricción importante que debe tener el formalismo G para que el teorema 2.5 se cumpla es la representación de $L_{0,1,d}$ como lenguaje generado por G sea $O(1)$. Puesto que el hecho de que la reducción anterior sea polinomial depende de que el tamaño de la representación del lenguaje $L_{0,1,d}$ a partir del formalismo G no dependa de la fórmula booleana F .

En este trabajo, al igual que en [10], se conjetura que el lenguaje $L_{0,1,d}$ tiene tamaño $O(1)$ en cualquiera de sus representaciones.

En la literatura consultada para este trabajo aparecen tres formalismos capaces de generar el lenguaje $L_{0,1,d}$: las gramáticas de índice global [4], las gramáticas de concatenación de rango [3] y las gramáticas libres de contexto múltiple paralelas (pMCFG) [12]. En todos los casos la gramática que genera el lenguaje $L_{0,1,d}$ tiene tamaño $O(1)$.

En el caso de las gramáticas de índice global, aplicando el teorema 2.5, el problema de intersección con lenguajes regulares es NP-duro, aun cuando en [4] se sugiere que este problema pueda ser resuelto en tiempo polinomial. Por otro lado, las gramáticas de concatenación de rango son cerradas bajo intersección con lenguajes regulares, pero el problema del vacío es indecidible, a menos que se cumplan ciertas restricciones, en cuyo caso no generan el lenguaje $L_{0,1,d}$ [3].

De acuerdo con la literatura consultada para este trabajo, las gramáticas libres de contexto múltiple paralelas satisfacen simultáneamente todas las propiedades requeridas: generan el lenguaje $L_{0,1,d}$ con representaciones de tamaño constante, son cerradas bajo intersección con lenguajes regulares y poseen problema del vacío decidable en tiempo polinomial. Teniéndose estas propiedades, de acuerdo con el teorema 2.5, sería posible resolver SAT en tiempo polinomial, representando $L_{0,1,d}$ con dicho formalismo.

Desafortunadamente, el procedimiento para determinar la intersección de una pMCFG con un lenguaje regular reportado por la literatura es incorrecto. En [12] se propone un algoritmo para resolver el problema para las gramáticas libres de contexto múltiple (MCFG), un caso particular de las pMCFG, y propone que este algoritmo pueda ser utilizado para las pMCFG. Sin embargo como se muestra en el próximo capítulo mediante un contraejemplo, dicho algoritmo no es correcto para las pMCFG, y, como se demuestra en el capítulo 4 determinar la intersección de una pMCFG con un lenguaje regular resulta ser un problema NP-duro.

En el siguiente capítulo se presentan las gramáticas libres de contexto múltiple paralelas.

Capítulo 3

Gramáticas libres de contexto múltiple paralelas

La gramáticas libres de contexto múltiple paralelas (pMCFG) [12] fueron introducidas en 1991 por Seki et al. como una generalización de las gramáticas libres de contexto (CFG). Estas fueron desarrolladas con el objetivo de proporcionar un formalismo gramatical capaz de generar lenguajes que no pueden ser generados por CFG, pero que aún mantienen ciertas de sus propiedades, como la decidibilidad del problema del vacío.

Las gramáticas libres de contexto múltiple paralelas se utilizan en el capítulo 4 para construir una gramática que genere el lenguaje de las interpretaciones de fórmulas booleanas.

En la sección 3.1 se presentan las definiciones formales de las pMCFG y de las gramáticas libres de contexto múltiple (MCFG), que son un caso particular de las pMCFG. En la sección 3.2 se muestra un algoritmo para resolver el problema del vacío de las pMCFG y se analiza su complejidad. En la sección 3.3 se expone el algoritmo para determinar la intersección de una MCFG con un lenguaje regular, se analiza su complejidad y se demuestra que dicho algoritmo no es aplicable a las pMCFG.

A continuación se introducen las gramáticas libres de contexto múltiple paralelas.

3.1. Definiciones

Las gramáticas libres de contexto múltiple (MCFG) y libres de contexto múltiple paralelas (pMCFG) son generalizaciones de las gramáticas libres de contexto (CFG) que permiten la generación de una familia de lenguajes más amplia que estas, pero más reducida que la de los lenguajes dependientes del contexto [12]. La idea de ambos formalismos es extender las CFG de forma tal que los símbolos no terminales puedan

generar tuplas de cadenas en lugar de una sola cadena [12].

Estas gramáticas contienen reglas de producción capaces de especificar cómo combinar tuplas de cadenas para generar nuevas tuplas, mediante la aplicación de una función intermedia que reordena y concatena las cadenas generadas por los símbolos no terminales.

En una CFG, una producción permite convertir una cadena de terminales y no terminales en otra. Por ejemplo la producción $A \rightarrow aB$ permite convertir la cadena $abAbb$ en la cadena $abaBbb$. En el caso de las pMCFG es posible convertir tuplas en tuplas, por ejemplo, la tupla (aaa,bbb,ccc) se pudiera convertir en la tupla $(ccca,bbba,aaac)$. Para ello se utilizan funciones de reordenamiento. Estas se definen a continuación.

Definición 3.1. Sea T un alfabeto. Una función f es una función de reordenamiento sobre T si existen un entero $k \geq 0$ y enteros $d_0, d_1, \dots, d_k \geq 0$ tales que

$$f : (T^*)^{d_1} \times \dots \times (T^*)^{d_k} \rightarrow (T^*)^{d_0},$$

y cada componente de

$$f(\mathbf{x}_1, \dots, \mathbf{x}_k)$$

es una concatenación finita de símbolos de T y de componentes de las tuplas

$$\mathbf{x}_i = (x_{i1}, \dots, x_{id_i})$$

para $1 \leq i \leq k$.

En la figura 3.1 se muestran ejemplos de funciones de reordenamiento. En los casos particulares de $g(\mathbf{x}_1, \mathbf{x}_2)$ y $h()$ se tiene que las componentes x_{ij} de las tuplas $\mathbf{x}_i$ aparecen a lo sumo una vez en las componentes de la tupla resultante de aplicar g y h . A las funciones de reordenamiento que cumplen esta condición se les llama funciones de reordenamiento lineales.

$$\begin{aligned} f : (T^*)^2 \times (T^*)^1 &\rightarrow (T^*)^3 & f[(x_{11}, x_{12}), (x_{21}, x_{22}, x_{23})] &= (x_{11}x_{21}x_{12}, x_{22}x_{23}x_{11}, aa x_{11}) \\ g : (T^*)^1 \times (T^*)^1 &\rightarrow (T^*)^1 & g[(x_{11}), (x_{21})] &= ax_{11}x_{21} \\ h : (T^*)^0 &\rightarrow (T^*)^1 & h() &= \varepsilon \end{aligned}$$

Figura 3.1: Ejemplos de funciones de reordenamiento sobre el alfabeto $T = \{a\}$.

Definición 3.2. Una función de reordenamiento $f(\mathbf{x}_1, \dots, \mathbf{x}_k)$ es lineal si cada componente x_{ij} de las tuplas $\mathbf{x}_i$ aparece a lo sumo una vez en las componentes de la tupla resultante de aplicar $f(\mathbf{x}_1, \dots, \mathbf{x}_k)$.

La función f de la figura 3.1 no es lineal, mientras que las funciones g y h sí lo son.

A partir de las funciones de reordenamiento se definen las gramáticas libres de contexto múltiple paralelas.

Definición 3.3. Una gramática libre de contexto múltiple paralela (pMCFG) es una tupla $G = (N, T, F, P, S)$ donde:

- N es un conjunto finito de símbolos llamados símbolos no terminales, cada $A \in N$ tiene una dimensión asociada $\dim(A) \geq 1$. Esta dimensión se representa mediante $\dim(A)$.
- T es un conjunto finito de símbolos llamados terminales. Se cumple que $N \cap T = \emptyset$.
- F es un conjunto finito de funciones de reordenamiento.
- P es un conjunto finito de reglas de la forma $A_0 \rightarrow f(A_1, \dots, A_k)$ con $k \geq 0, f \in F$ tal que $f : (T^*)^{\dim(A_1)} \times \dots \times (T^*)^{\dim(A_k)} \rightarrow (T^*)^{\dim(A_0)}$.
- S es un símbolo no terminal inicial con $\dim(S) = 1$.

La dimensión de los no terminales está determinada por las reglas de producción en las que participan. En una regla de la forma

$$A_0 \rightarrow f(A_1, \dots, A_k),$$

la función de reordenamiento

$$f : (T^*)^{\dim(A_1)} \times \dots \times (T^*)^{\dim(A_k)} \rightarrow (T^*)^{\dim(A_0)}$$

impone que el símbolo A_i genere exactamente $\dim(A_i)$ componentes, mientras que A_0 genera $\dim(A_0)$ componentes.

Por ejemplo, si se tiene una función de reordenamiento

$$f : (T^*)^2 \times (T^*)^1 \rightarrow (T^*)^3,$$

entonces una regla

$$A \rightarrow f(B, C)$$

requiere que $\dim(B) = 2$, $\dim(C) = 1$ y $\dim(A) = 3$.

Los símbolos terminales corresponden a los símbolos ordinarios de las cadenas generadas por la gramática. Por ejemplo, si $T = \{a, b\}$, entonces las cadenas producidas por la gramática estarán formadas únicamente por símbolos a y b .

Asimismo, las funciones de reordenamiento especifican cómo combinar las componentes generadas por distintos no terminales. Por ejemplo, las funciones

$$\begin{aligned} f : (T^*)^1 &\rightarrow (T^*)^1 & f[(x_{11})] &= x_{11}x_{11} \\ g : (T^*)^1 \times (T^*)^1 &\rightarrow (T^*)^1 & g[(x_{11}), (x_{21})] &= ax_{11}x_{21} \end{aligned}$$

duplican una cadena y concatenan dos cadenas precedidas por el símbolo terminal a , respectivamente.

En una pMCFG cada símbolo no terminal A tiene asociado un conjunto $L(A)$ de tuplas de cadenas, construido aplicando las funciones de reordenamiento a tuplas previamente obtenidas a partir de las reglas de la pMCFG. El lenguaje generado por la pMCFG es el conjunto de cadenas que se pueden obtener a partir del símbolo inicial S mediante este proceso.

Definición 3.4. Sea $G = (N, T, F, P, S)$ una pMCFG. Se define para todo $A \in N$, el lenguaje asociado $L(A)$ donde $L(A) \subseteq (T^*)^{\dim(A)}$ como el menor conjunto que cumple las siguientes condiciones:

- Para toda regla $A \rightarrow f() \in P$, se tiene que $f() \in L(A)$.
- Para toda regla $A \rightarrow f(A_1, \dots, A_k) \in P$ con $k \geq 1$, si $\mathbf{x}_i \in L(A_i)$ para $1 \leq i \leq k$, entonces se tiene que $f(\mathbf{x}_1, \dots, \mathbf{x}_k) \in L(A)$.

El lenguaje generado por la pMCFG G es el conjunto de cadenas terminales asociado al símbolo inicial S , es decir, $L(G) = \{w \in T^* \mid (w) \in L(S)\}$.

Nótese que los elementos del lenguaje de un símbolo no terminal A son tuplas de cadenas, mientras que los elementos del lenguaje generado por la pMCFG son cadenas de símbolos terminales. Debido a que el símbolo inicial S tiene dimensión 1, se tiene que $L(S)$ es un conjunto de tuplas de la forma (w) con $w \in T^*$. Al lenguaje de una pMCFG pertenecen las cadenas w para las cuales existe una tupla $(w) \in L(S)$, no exactamente las tuplas (w) que pertenecen a $L(S)$.

La pMCFG G_1 del ejemplo 3.2 de la página 34 genera el lenguaje $\{a^{2^n} \mid n \geq 0\}$ [12]. Para ilustrar este hecho se demostrará que la cadena $a^4 \in L(G_1)$:

- $(\varepsilon) \in L(A)$ porque existe la producción $A \rightarrow h()$ y $h() = \varepsilon$.
- $(a) \in L(S)$ porque $(\varepsilon) \in L(A)$, $g[(\varepsilon), (\varepsilon)] = (a)$ y existe la producción $S \rightarrow g(A, A)$.
- $(a^2) \in L(S)$ porque $(a) \in L(S)$, $f[(a)] = (aa)$ y existe la producción $S \rightarrow f(S)$.
- $(a^4) \in L(S)$ porque $(a^2) \in L(S)$, $f[(a^2)] = (a^4)$ y existe la producción $S \rightarrow f(S)$.

$G_1 = (N, T, F, P, S)$ donde $N = \{S, A\}$ y $T = \{a\}$.
 P y F contienen las siguientes producciones y funciones respectivamente:

$$\begin{array}{ll} S \rightarrow f(S), & f[(x_{11})] = (x_{11}x_{11}) \\ S \rightarrow g(A, A), & g[(x_{11}), (x_{21})] = (ax_{11}x_{21}) \\ A \rightarrow h(), & h() = (\varepsilon) \end{array}$$

Figura 3.2: Gramática libre de contexto múltiple paralela que genera el lenguaje $\{a^{2^n} \mid n \geq 0\}$.

- $a^4 \in L(G_1)$ porque $(a^4) \in L(S)$.

En [12] se definen las gramáticas libres de contexto múltiple (MCFG) las cuales son un caso particular de las pMCFG, en las que se restringe el tipo de funciones de reordenamiento permitidas a funciones lineales. Este formalismo es de interés para este trabajo, pues en [12] se presenta un método para construir la intersección de una MCFG con un lenguaje regular el cual se propone también para pMCFG. Este método se expone en la sección 3.3 y se demuestra que no es aplicable para las pMCFG.

A continuación se definen formalmente las gramáticas libres de contexto múltiple.

Definición 3.5. Una gramática libre de contexto múltiple (MCFG) es una tupla $G = (N, T, F, P, S)$ donde:

- N es un conjunto finito de símbolos llamados símbolos no terminales, cada $A \in N$ tiene una dimensión asociada $\dim(A) \geq 1$. Esta dimension se representa mediante $\dim(A)$.
- T es un conjunto finito de símbolos llamados terminales. Se cumple que

$$N \cap T = \emptyset.$$

- F es un conjunto finito de funciones de reordenamiento lineales.
- P es un conjunto finito de reglas de la forma $A_0 \rightarrow f(A_1, \dots, A_k)$ con $k \geq 0$, $f \in F$ tal que $f : (T^*)^{\dim(A_1)} \times \dots \times (T^*)^{\dim(A_k)} \rightarrow (T^*)^{\dim(A_0)}$.
- S es un símbolo no terminal inicial con $\dim(S) = 1$.

El lenguaje generado por una MCFG se define de la misma manera que el lenguaje generado por una pMCFG.

En el ejemplo de la figura 3.2 de la página 34, la pMCFG G_1 no es una MCFG ya que la función f no es lineal.

En la figura 3.3 se muestra la pMCFG G_2 que genera el lenguaje $\{www \mid w \in \{0,1\}^+\}$ [8]. Dado que Las funciones f_1, f_2, f_3, f_4, f_5 son lineales se tiene que G_2 es una MCFG. Para ilustrar este hecho se demostrará que la cadena $010101 \in L(G_2)$:

- $(1,1,1) \in L(A)$ porque existe la producción $A \rightarrow f_5()$ y $f_5() = (1,1,1)$.
- $(01,01,01) \in L(A)$ porque $(1,1,1) \in L(A)$, $f_2[(1,1,1)] = (01,01,01)$ y existe la producción $A \rightarrow f_2(A)$.
- $(010101) \in L(S)$ porque $(01,01,01) \in L(A)$, $f_1[(01,01,01)] = (010101)$ y existe la producción $S \rightarrow f_1(A)$.
- $010101 \in L(G_2)$ porque $(010101) \in L(S)$.

$$G_2 = (N, T, F, P, S) \text{ donde } N = \{S, A\} \text{ y } T = \{0, 1\}.$$

P y F contienen las siguientes producciones y funciones respectivamente:

$S \rightarrow f_1(A)$	$f_1[(x, y, z)] = (xyz)$
$A \rightarrow f_2(A)$	$f_2[(x, y, z)] = (0x, 0y, 0z)$
$A \rightarrow f_3(A)$	$f_3[(x, y, z)] = (1x, 1y, 1z)$
$A \rightarrow f_4()$	$f_4() = (0, 0, 0)$
$A \rightarrow f_5()$	$f_5() = (1, 1, 1)$

Figura 3.3: Gramática libre de contexto múltiple paralela que genera el lenguaje $\{www \mid w \in \{0,1\}^+\}$.

En siguientes secciones se estudia cómo resolver el problema del vacío de las pMCFG y la intersección de estas con lenguajes regulares.

3.2. Problema del vacío para pMCFG y MCFG

Aunque la familia de lenguajes que generan las pMCFG es estrictamente más amplia que la de las CFG [12], resolver el problema del vacío para una pMCFG es igual de complejo que resolver el mismo problema para una CFG. De hecho, el algoritmo para resolver el problema del vacío para una pMCFG es similar al algoritmo análogo para una CFG expuesto en [6].

Para exponer el algoritmo que determina si el lenguaje generado por una pMCFG es vacío, es necesario definir la noción de símbolo no terminal generador en una pMCFG.

Definición 3.6. Sea $G = (N, T, F, P, S)$ una pMCFG. Un símbolo no terminal $A \in N$ es generador si y solo si el conjunto asociado al no terminal A es no vacío. Formalmente

$$L(A) \neq \emptyset.$$

Por ejemplo, en la gramática G_3 de la figura 3.4 el símbolo no terminal S es generador, ya que $L(S) = \{(a)\}$ no es vacío, mientras que el símbolo no terminal A no es generador, ya que $L(A)$ es un conjunto vacío.

$G_3 = (N, T, F, P, S)$ donde $N = \{S, A\}$ y $T = \{a, b\}$.
 P y F contienen las siguientes producciones y funciones respectivamente:

$$\begin{array}{ll} S \rightarrow g(), & g() = (a) \\ A \rightarrow f(A) & f[(x)] = (x) \end{array}$$

Figura 3.4: Gramática libre de contexto múltiple paralela con símbolos generadores y no generadores.

Como consecuencia directa, para una pMCFG G con símbolo inicial S se tiene que $L(G)$ contiene algún elemento si y solo si S es generador. Esto permite que determinar si $L(G)$ es vacío, sea equivalente a computar el conjunto de no terminales generadores de G y comprobar si S no pertenece a este conjunto.

El algoritmo para determinar si $L(G)$ es vacío consiste en construir iterativamente el conjunto de símbolos generadores.

Inicialmente, se consideran generadores todos los símbolos no terminales que poseen una regla de producción de la forma $A \rightarrow f()$. Luego, mientras existan reglas $A \rightarrow f(A_1, \dots, A_k)$ tales que todos los símbolos A_i sean generadores y A no lo sea, se añade A al conjunto de símbolos generadores.

Este procedimiento replica directamente la definición 3.4 en la que se define el conjunto $L(A)$, ya que un símbolo no terminal es generador si y solo si puede obtenerse a partir de reglas cuyos argumentos generan tuplas terminales.

Algoritmo 3.1: Determinación de símbolos generadores en una pMCFG

Entrada: Una pMCFG $M = (N, T, F, P, S)$
Salida: Conjunto de símbolos generadores

```

1  $Gen \leftarrow \{A \in N \mid A \rightarrow f() \in P\};$ 
2  $cambio \leftarrow \text{verdadero};$ 
3 mientras  $cambio$  hacer
4    $cambio \leftarrow \text{falso};$ 
5   para cada regla  $A \rightarrow f(A_1, \dots, A_k) \in P$  hacer
6     si  $A_1, \dots, A_k \in Gen$  y  $A \notin Gen$  entonces
7       añadir  $A$  a  $Gen$ ;
8        $cambio \leftarrow \text{verdadero};$ 
9     fin
10  fin
11 fin
12 devolver  $Gen$ ;
```

A continuación se procederá a analizar la correctitud y complejidad del algoritmo presentado.

La correctitud del algoritmo es directa dada la definición de $L(A)$: cada símbolo añadido al conjunto corresponde a un símbolo no terminal capaz de generar al menos una tupla terminal, y recíprocamente, todo símbolo generador será eventualmente añadido por el algoritmo. Por lo tanto, al finalizar la ejecución del algoritmo, el conjunto calculado coincide exactamente con el conjunto de símbolos generadores de la gramática. En consecuencia, el lenguaje generado por G no es vacío si y solo si S es generador. Esto es

$$L(G) \neq \emptyset \iff S \in Gen.$$

Para analizar la complejidad del algoritmo, se definirá el tamaño $|G|$ de una pMCFG G como el número total de ocurrencias de símbolos no terminales en las reglas de producción de G . Por ejemplo, la gramática G_1 del ejemplo 3.2 de la página 34 tiene tamaño $|G_1| = 3$.

En cada iteración del ciclo, el algoritmo recorre todas las reglas de producción y, para cada regla $A \rightarrow f(A_1, \dots, A_k)$ verifica si todos los símbolos $A_1, \dots, A_k$ pertenecen a Gen . Por lo tanto, el costo total de cada iteración es lineal en el tamaño de la gramática.

Además, en cada iteración se añade al menos un símbolo no terminal nuevo a Gen , o el algoritmo termina. Como existen a lo sumo $|N|$ símbolos no terminales, el número total de iteraciones es acotado por $|N|$.

En consecuencia, el algoritmo tiene complejidad temporal $O(|N| \cdot |G|)$, y, dado que $|N| \leq |G|$, también puede expresarse como $O(|G|^2)$. Este resultado permite enunciar el siguiente teorema:

Teorema 3.1. *El problema del vacío para una pMCFG es decidible en tiempo polinomial respecto al tamaño de la gramática.*

Aunque el algoritmo presentado ya tiene complejidad polinomial, este puede optimizarse de manera análoga al algoritmo para determinar los símbolos generadores en una CFG [6]. A continuación se describe la idea de esta optimización.

En lugar de recorrer repetidamente todas las reglas de producción, es posible mantener, para cada regla, la cantidad de símbolos no terminales de su lado derecho que aún no han sido identificados como generadores. Cada vez que un nuevo símbolo se añade a Gen , se actualizan únicamente las reglas que dependen de este símbolo. De esta forma, cada regla y cada ocurrencia de un símbolo no terminal se procesan un número constante de veces. Por lo tanto, el problema del vacío para una pMCFG puede resolverse en tiempo lineal respecto al tamaño de la gramática [12].

En la siguiente sección se analiza la intersección de una pMCFG y una MCFG con un lenguaje regular, y se demuestra que el método para construir la intersección de una MCFG con un lenguaje regular no es aplicable a las pMCFG.

3.3. Intersección de pMCFG y MCFG con lenguajes regulares

En el capítulo 2 se demuestra que es posible construir el lenguaje de las interpretaciones que satisfacen una fórmula intersectando un autómata finito determinista con el lenguaje $L_{0,1,d}$. Esto permite que sea posible resolver el problema de la satisfacibilidad booleana decidiendo si el lenguaje resultante de dicha intersección es vacío o no. Dado que el problema del vacío para una pMCFG es decidible en tiempo polinomial, es de interés estudiar la intersección de una pMCFG o una MCFG con un lenguaje regular, y analizar si el resultado de esa intersección pertenece a la misma familia de lenguajes.

3.3.1. Intersección de MCFG con DFA

En [12] se demuestra que la intersección de una MCFG con un lenguaje regular es una MCFG. La idea de la demostración es la siguiente.

Dada una MCFG $G = (N, T, F, P, S)$ y un DFA $\mathcal{A} = (Q, T, \delta, q_0, F_{\mathcal{A}})$ que reconoce un lenguaje regular R , se construye una nueva MCFG G' tal que $L(G') = L(G) \cap R$.

La idea central consiste en enriquecer cada símbolo no terminal A de la MCFG con información sobre los estados del DFA asociados a las componentes de las tuplas en $L(A)$. Si A tiene dimensión d , entonces los nuevos símbolos no terminales tienen la forma

$$A[(p_1, q_1), \dots, (p_d, q_d)]$$

con $p_i, q_i \in Q$ para cada $1 \leq i \leq d$.

La idea es que este símbolo represente la restricción de A a aquellas tuplas cuya i -ésima componente lleva al DFA desde el estado p_i al estado q_i .

Formalmente, la construcción garantiza la siguiente propiedad:

$$\mathbf{w} \in L(A[(p_1, q_1), \dots, (p_d, q_d)]) \iff \mathbf{w} \in L(A) \text{ y } \hat{\delta}(p_i, w_i) = q_i \quad \forall 1 \leq i \leq d, \quad (3.1)$$

donde $\hat{\delta}$ es la función de transición extendida del DFA y $\mathbf{w} = (w_1, \dots, w_d)$.

Por cada regla $A_0 \rightarrow f(A_1, \dots, A_k)$ de G se agrega un conjunto de reglas a en G' , una por cada elección de estados para $A_0, A_1, \dots, A_k$ que sea consistente con f .

Para precisar qué significa «consistente», considérese una regla $A_0 \rightarrow f(A_1, \dots, A_k)$ y una elección de no-terminales enriquecidos del lado derecho:

$$A'_i = A_i[(p_1^{(i)}, q_1^{(i)}), \dots, (p_{d_i}^{(i)}, q_{d_i}^{(i)})], \quad 1 \leq i \leq k$$

Cada componente j de f es una concatenación de la forma:

$$\alpha_0 z_1 \alpha_1 z_2 \cdots z_m \alpha_m,$$

donde cada α_l es una cadena de símbolos terminales (posiblemente vacía) y cada z_l es una variable que referencia a una componente de algún A_i . Como ya se fijaron los pares de estados de cada A'_i , cada variable z_l tiene asociados un estado inicial r_l y un estado final s_l : los del par correspondiente.

La componente j del A'_0 en el lado izquierdo, tendrá asociado un par $(p_j^{(0)}, q_j^{(0)})$. Se dice que esta componente es «compatible» si se cumplen las siguientes condiciones:

- $\hat{\delta}(p_j^{(0)}, \alpha_0) = r_1$: leer α_0 desde el estado inicial de la componente lleva al estado inicial de la primera variable.
- $\hat{\delta}(s_l, \alpha_l) = r_{l+1}$ para cada $1 \leq l < m$: el estado en el que queda el DFA tras leer la variable z_l debe conectar, mediante α_l , con el estado inicial de la variable z_{l+1} .
- $\hat{\delta}(s_m, \alpha_m) = q_j^{(0)}$: tras leer la última variable y el último tramo de terminales, el DFA debe quedar en el estado final de la componente.

La regla $A'_0 \rightarrow f(A'_1, \dots, A'_k)$ se añade a G' si todas sus componentes son compatibles simultáneamente.

Finalmente, por cada estado de aceptación $q_f \in F_{\mathcal{A}}$, se añade una nueva regla de la forma $S' \rightarrow S[(q_0, q_f)]$ que exige que la cadena generada lleve al DFA desde su estado inicial q_0 hasta un estado de aceptación q_f . De esta forma, la gramática construida genera exactamente el lenguaje $L(G) \cap R$, donde G es la MCFG original y R es el lenguaje reconocido por el DFA.

Como consecuencia, se tiene el siguiente teorema.

Teorema 3.2. *La familia de lenguajes generados por las MCFG es cerrada bajo intersección con lenguajes regulares. Esto es, si G es una MCFG y R es un lenguaje regular, entonces existe una MCFG G' tal que $L(G') = L(G) \cap R$.*

Demostración Sea una MCFG G y un lenguaje regular R , sea un DFA $\mathcal{A}$ que reconoce R . Se construye una MCFG G' tal que $L(G') = L(G) \cap R$ siguiendo el procedimiento de [12] descrito anteriormente y se obtiene una MCFG G' tal que $L(G') = L(G) \cap L(\mathcal{A})$. Dado que $L(\mathcal{A}) = R$, se concluye que $L(G') = L(G) \cap R$. $\square$

La construcción anterior incrementa el número de símbolos no terminales y reglas de producción al considerar todas las combinaciones posibles de estados del DFA asociadas a las componentes de cada no terminal. El siguiente teorema acota el tamaño de la MCFG resultante y el costo de su construcción.

Teorema 3.3. *Sean $G = (N, T, F, P, S)$ una MCFG y $\mathcal{A} = (Q, T, \delta, q_0, F_{\mathcal{A}})$ un DFA, sean además:*

- $q = |Q|$ el número de estados del $\mathcal{A}$,
- $D = \max_{A \in N} \dim(A)$ la máxima dimensión de un símbolo no terminal de G .
- $r = \max\{k \mid A_0 \rightarrow f(A_1, \dots, A_k) \in P\}$ el número máximo de símbolos no terminales en el lado derecho de una regla.
- V el máximo número de variables que aparecen en una componente de una función de reordenamiento.
- T la máxima longitud de las cadenas de símbolos terminales insertadas entre variables en las funciones de reordenamiento.

Entonces la MCFG G' obtenida por la construcción anterior tiene $O(|P| \cdot q^{2(r+1)D})$ reglas de producción, y puede construirse en tiempo $O(|P| \cdot q^{2(r+1)D} \cdot DVT)$.

Demostración Si un símbolo no terminal A tiene dimensión d , entonces los símbolos enriquecidos asociados a A tienen la forma $A[(p_1, q_1), \dots, (p_d, q_d)]$, donde cada $p_i, q_i \in Q$. Por lo tanto, existen q^{2d} versiones enriquecidas posibles de A . En particular, el número de símbolos no terminales construidos para un símbolo de dimensión a lo sumo D es $O(q^{2D})$, aunque, en la práctica, muchos de estos símbolos no se utilizan en reglas de la nueva gramática.

Considérese ahora una regla de producción $A_0 \rightarrow f(A_1, \dots, A_k)$. Para construir las reglas correspondientes en la nueva gramática, es necesario considerar todas las combinaciones posibles de estados asociados a los símbolos $A_0, A_1, \dots, A_k$. Como

$$\sum_{i=0}^k \dim(A_i) \leq (k+1)D \leq (r+1)D,$$

entonces el número de combinaciones posibles de estados es

$$q^{2 \sum_{i=0}^k \dim(A_i)} \leq q^{2(r+1)D}.$$

Por lo tanto, en el peor caso una regla original genera $O(q^{2(r+1)D})$ reglas en la nueva gramática, y el número total de reglas en G' es $O(|P| \cdot q^{2(r+1)D})$.

Además, por cada combinación de estados, es necesario verificar la compatibilidad de las transiciones del DFA con la concatenación definida por la función de reordenamiento correspondiente. La verificación de la compatibilidad para una regla cuesta $O(DVT)$, ya que a lo sumo deben verificarse D componentes, cada una con $O(V)$ variables y $O(T)$ símbolos terminales entre ellas.

En total el costo de la construcción es $O(|P| \cdot q^{2(r+1)D} \cdot DVT)$. $\square$

Aunque la construcción anterior puede tomar un tiempo exponencial respecto a la gramática original, es polinomial respecto al número de estados del DFA. Esto significa que para una MCFG específica, intersectar dicha MCFG con un DFA es un problema que puede ser resuelto en tiempo polinomial.

La construcción del teorema 3.2 no es aplicable en el caso de las pMCFG, ya que para mantener la propiedad de la ecuación 3.1 es necesario que las funciones de reordenamiento sean lineales.

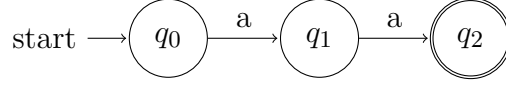
En la siguiente sección se muestra que el método propuesto para hallar la intersección de una MCFG con un DFA no es suficiente para hallar la intersección de una pMCFG con un DFA.

3.3.2. Intersección de pMCFG con DFA

Hallar la intersección de una pMCFG con un DFA es más complejo que en el caso de una MCFG, debido a que las funciones de reordenamiento no son necesariamente lineales. En [12] se plantea que se puede usar el mismo método que para una MCFG. Sin embargo, este método no es aplicable a una pMCFG, es posible demostrarlo con un contraejemplo.

Sea la pMCFG $G_{aa} = (N, T, F, P, S)$ con

- $N = \{S, A\}$
- $T = \{a\}$
- $F = \{f, g\}$ donde $f(x) = xx$ y $g() = a$.
- P contiene las reglas de producción $S \rightarrow f(A)$ y $A \rightarrow g()$.

Figura 3.5: DFA $\mathcal{A}$ que reconoce el lenguaje $\{aa\}$.

Dada la definición de $L(G_{aa})$, se tiene que:

- $(a) \in L(A)$ porque existe la producción $A \rightarrow g()$ y $g() = (a)$.
- $(aa) \in L(S)$ porque $(a) \in L(A)$, $f[(a)] = (aa)$ y existe la producción $S \rightarrow f(A)$.
- $aa \in L(G_{aa})$ porque $(aa) \in L(S)$.

Luego lo tanto $aa \in L(G_{aa})$.

Sea $\mathcal{A} = (\{q_0, q_1, q_2\}, T, \delta, q_0, \{q_2\})$ con función de transición δ definida como:

- $\delta(q_0, a) = q_1$
- $\delta(q_1, a) = q_2$

Este autómata reconoce el lenguaje $\{aa\}$, por lo tanto $L(\mathcal{A}) = \{aa\}$. En la figura 3.5 se muestra la representación gráfica del autómata $\mathcal{A}$.

Se tiene que la cadena aa pertenece a $L(G_{aa}) \cap L(\mathcal{A})$, por tanto al efectuar la construcción del teorema 3.2 para hallar la intersección, la gramática obtenida debería ser capaz de reconocer aa .

Al efectuar el método propuesto. La nueva gramática $G' = (N', T, F, P', S')$ tiene los símbolos no terminales

$$N' = \{S'\} \cup \{A[(p, q)], S[(p, q)] \mid p, q \in Q\}$$

En las nuevas reglas de producción P' las reglas correspondientes a $A \rightarrow g()$ son de la forma

$$A[(p, q)] \rightarrow g().$$

Dado que $g() = a$, las reglas de esta forma solo serán válidas si

$$\delta(p, a) = q,$$

lo cual solo se cumple para (q_0, q_1) y (q_1, q_2) . Luego, las reglas de esta forma en P' son

$$A[(q_0, q_1)] \rightarrow g() \quad \text{y} \quad A[(q_1, q_2)] \rightarrow g().$$

Por tanto, solo los no terminales $A[(q_0, q_1)]$ y $A[(q_1, q_2)]$ pueden generar la cadena a .

Las reglas en P' correspondientes a $S \rightarrow f(A)$ con $f(x) = xx$, son de la forma

$$S[(p, q)] \rightarrow f(A[(r, s)]).$$

La única componente de f es xx , la cual es, una concatenación de la forma $\alpha_0 z_1 \alpha_1 z_2 \alpha_2$ con

$$\alpha_0 = \alpha_1 = \alpha_2 = \varepsilon \quad \text{y} \quad z_1 = z_2 = x.$$

Como ambas ocurrencias de x hacen referencia al mismo no terminal $A[(r, s)]$ ambas tienen asociado el mismo par de estados (r, s) .

Aplicando las condiciones de compatibilidad de la construcción:

- $\hat{\delta}(p, \alpha_0) = r$: como $\alpha_0 = \varepsilon$, entonces $p = r$.
- $\hat{\delta}(s, \alpha_1) = r$: como $\alpha_1 = \varepsilon$, y el estado final tras leer z_1 es s , entonces $s = r$.
- $\hat{\delta}(s, \alpha_2) = q$: como $\alpha_2 = \varepsilon$, y el estado final tras leer z_2 es s , entonces $s = q$.

De las igualdades anteriores se deduce que $p = r = s = q$. Por lo tanto las únicas reglas válidas en P' asociadas a $S \rightarrow f(A)$ son de la forma

$$S[(p, p)] \rightarrow f(A[(p, p)]).$$

Para que $S[(p, p)]$ genere alguna cadena, el no terminal $A[(p, p)]$ debe poder generar alguna cadena. Pero según el análisis anterior, los únicos $A[(r, s)]$ generadores son $A[(q_0, q_1)]$ y $A[(q_1, q_2)]$, ambos con $r \neq s$. Por lo tanto, ningún $A[(p, p)]$ es generador, y por ende ningún $S[(p, p)]$ lo es.

Por lo tanto, la gramática obtenida no puede generar la cadena aa , lo cual demuestra que el método propuesto no es correcto para hallar la intersección de una pMCFG con un DFA.

En este capítulo se introdujeron las pMCFG y se demostró que el método propuesto para hallar la intersección de una MCFG con un DFA no es correcto para hallar la intersección de una pMCFG con un DFA. En el siguiente capítulo se aplica el teorema 2.5 para demostrar que de existir un algoritmo para obtener una pMCFG que reconozca esta intersección este es polinomial solo si $P = NP$.

Capítulo 4

Lenguaje de las interpretaciones que satisfacen una fórmula empleando gramáticas libres de contexto múltiple

En esta sección se propone una gramática libre de contexto múltiple paralela (pMCFG) que genera el lenguaje de las interpretaciones de fórmulas booleanas codificadas como cadenas binarias separadas por el símbolo d . Esta gramática se utilizará junto con el teorema 2.5 del capítulo 2 para demostrar que cualquier algoritmo que construya la intersección de una pMCFG con un lenguaje regular no puede ser polinomial a menos que $P = NP$.

En la sección 4.1 se presenta la gramática pMCFG que genera el lenguaje de las interpretaciones de fórmulas booleanas codificadas como cadenas binarias separadas por el símbolo d , se explica su funcionamiento y se demuestra formalmente que esta genera el lenguaje $L_{0,1,d}$. En la sección 4.2 se utiliza esta gramática para demostrar que si las pMCFG fueran cerradas bajo intersección con lenguajes regulares, hallar la intersección entre una pMCFG y un lenguaje regular es un problema NP-duro.

A continuación se muestra que el lenguaje $L_{0,1,d}$ puede ser descrito por una pMCFG.

4.1. $L_{0,1,d}$ como lenguaje de múltiple contexto paralelo

En esta sección se presenta una gramática libre de contexto múltiple paralela (pMCFG) que reconoce el lenguaje:

$$L_{0,1,d} = \{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\},$$

la cual puede utilizarse para representar las interpretaciones de una fórmula booleana codificadas como secuencias binarias separadas por el símbolo d .

La idea de la construcción es dividir el proceso de generación en dos etapas. Primero se genera una cadena de la forma wd , donde w es una cadena binaria cualquiera. Luego, esta misma cadena se repite una o más veces mediante una acumulación progresiva sobre una segunda componente de la tupla generada.

Para ello se define la gramática pMCFG $G_{0,1,d}$ de la figura 4.1.

$G_{0,1,d} = (N, T, F, P, S)$ donde $N = \{S, C, A\}$ y $T = \{0, 1, d\}$.
 P y F contienen las siguientes producciones y funciones respectivamente:

$S \rightarrow res(C)$	$res[(x, y)] = (y)$
$C \rightarrow copy(C)$	$copy[(x, y)] = (x, xy)$
$C \rightarrow copy(A)$	
$A \rightarrow f_0(A)$	$f_0[(x, y)] = (0x, y)$
$A \rightarrow f_1(A)$	$f_1[(x, y)] = (1x, y)$
$A \rightarrow g()$	$g() = (d, \varepsilon)$

Figura 4.1: Gramática pMCFG $G_{0,1,d}$ que genera el lenguaje $L_{0,1,d}$.

El comportamiento de la gramática puede describirse de la siguiente forma:

- El símbolo no terminal A genera tuplas de la forma (wd, ε) . La primera componente contiene una cadena binaria terminada en d , mientras que la segunda componente permanece vacía.
- El símbolo no terminal C utiliza la función $copy$ para copiar una cantidad arbitraria $n \geq 1$ de veces la primera componente sobre la segunda. De esta forma se generan tuplas de la forma $(wd, (wd)^n)$ con $n \geq 1$.
- Finalmente, el símbolo inicial S utiliza la función res para extraer la segunda componente de la tupla generada por C , obteniéndose una cadena de la forma $(wd)^n$ para algún $n \geq 1$.

Para ilustrar el funcionamiento de la gramática $G_{0,1,d}$, se demostrará que $010d010d \in L(G_{0,1,d})$:

1. $(d, \varepsilon) \in L(A)$ porque

- $g() = (d, \varepsilon)$
 - y existe la producción $A \rightarrow g()$.
2. $(0d, \varepsilon) \in L(A)$ porque
- $(d, \varepsilon) \in L(A)$,
 - $f_0[(d, \varepsilon)] = (0d, \varepsilon)$
 - y existe la producción $A \rightarrow f_0(A)$.
3. $(10d, \varepsilon) \in L(A)$ porque
- $(0d, \varepsilon) \in L(A)$,
 - $f_1[(0d, \varepsilon)] = (10d, \varepsilon)$
 - y existe la producción $A \rightarrow f_1(A)$.
4. $(010d, \varepsilon) \in L(A)$ porque
- $(10d, \varepsilon) \in L(A)$,
 - $f_0[(10d, \varepsilon)] = (010d, \varepsilon)$.
 - y existe la producción $A \rightarrow f_0(A)$.
5. $(010d, 010d) \in L(C)$ porque
- $(010d, \varepsilon) \in L(A)$,
 - $copy[(010d, \varepsilon)] = (010d, 010d)$
 - y existe la producción $C \rightarrow copy(A)$.
6. $(010d, 010d010d) \in L(C)$ porque
- $(010d, 010d) \in L(C)$,
 - $copy[(010d, 010d)] = (010d, 010d010d)$
 - y existe la producción $C \rightarrow copy(C)$.
7. $(010d010d) \in L(S)$ porque
- $(010d, 010d010d) \in L(C)$,
 - $res[(010d, 010d010d)] = (010d010d)$
 - y existe la producción $S \rightarrow res(C)$.
8. $010d010d \in L(G_{0,1,d})$ porque $(010d010d) \in L(S)$ y S es el símbolo inicial de $G_{0,1,d}$.

En los primeros 4 pasos se tiene la cadena $wd = 010d$ en la primera componente de la tupla. Luego, se acumula ese resultado en la segunda componente en los siguientes 2 pasos aplicando la función *copy*. Finalmente, se aplica la función *res* para extraer la cadena acumulada resultante. Como $(010d010d) \in L(S)$ entonces $010d010d \in L(G_{0,1,d})$.

El la gramática $G_{0,1,d}$ genera exactamente el lenguaje $L_{0,1,d}$. Este resultado se presenta en el siguiente teorema.

Teorema 4.1. Sean $G_{0,1,d}$ la gramática *pMCFG* definida en la figura 4.1 y $L_{0,1,d}$ el lenguaje definido como

$$\{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\}$$

Se cumple que $L(G_{0,1,d}) = L_{0,1,d}$.

En las siguientes secciones se demostrará formalmente el teorema 4.1. Para ello se presentan tres lemas con el objetivo de caracterizar el lenguaje asociado a cada uno de los símbolos no terminales de la gramática.

A continuación se analizará el lenguaje asociado al no terminal A .

Lenguaje asociado al no terminal A

El símbolo no terminal A se utiliza para construir la cadena base wd . La regla

$$A \rightarrow g()$$

introduce el símbolo terminal d , mientras que las reglas:

$$A \rightarrow f_0(A), \quad A \rightarrow f_1(A)$$

permiten agregar símbolos binarios al inicio de la primera componente de la tupla. Como consecuencia, A genera exactamente las tuplas cuya primera componente es una cadena binaria terminada en d , y cuya segunda componente es la cadena vacía. Esta idea se formaliza en el siguiente lema.

Lema 4.1. El lenguaje asociado por el no terminal A es:

$$L(A) = \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}.$$

Demostración Primero se demostrará por inducción sobre la longitud de w que toda tupla de la forma (wd, ε) con $w \in \{0,1\}^*$ pertenece al lenguaje asociado a A , esto es

$$\{(wd, \varepsilon) \mid w \in \{0,1\}^*\} \subseteq L(A).$$

Caso base $|w| = 0$. Se tiene que $w = \varepsilon$, luego $wd = d$ y $(d, \varepsilon) \in L(A)$ ya que $A \rightarrow g()$.

Paso inductivo Supóngase que para todo $w' \in \{0,1\}^*$ con $|w'| = k$ se cumple que $(w'd, \varepsilon) \in L(A)$.

Entonces, para cualquier cadena de la forma $aw'd$ se tiene que $(aw'd, \varepsilon) \in L(A)$ con $a \in \{0,1\}$, ya que $(w'd, \varepsilon) \in L(A)$, $f_a[(w'd, \varepsilon)] = (aw'd, \varepsilon)$ y se tiene la regla $A \rightarrow f_a(A)$. Luego $(wd, \varepsilon) \in L(A)$ para todo $w \in \{0,1\}^*$.

A continuación se demostrará que

$$L(A) \subseteq \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$$

Por definición, $L(A)$ es el menor conjunto de tuplas en $(T^*)^2$ que cumple las siguientes condiciones:

- $(d, \varepsilon) \in L(A)$ por la regla $A \rightarrow g()$.
- Si $(x, y) \in L(A)$, entonces $f_0[(x, y)] \in L(A)$ por la regla $A \rightarrow f_0(A)$.
- Si $(x, y) \in L(A)$, entonces $f_1[(x, y)] \in L(A)$ por la regla $A \rightarrow f_1(A)$.

En efecto, $\{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$ cumple las condiciones anteriores:

- (d, ε) es de la forma (wd, ε) con $w = \varepsilon$, luego $(d, \varepsilon) \in \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$.
- Si $(x, y) \in \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$ entonces $(x, y) = (wd, \varepsilon)$ para algún $w \in \{0,1\}^*$. Luego $f_0[(x, y)] = (0wd, \varepsilon)$ es de la forma $(w'd, \varepsilon)$ con $w' = 0w \in \{0,1\}^*$, luego $f_0[(x, y)] \in \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$.
- Si $(x, y) \in \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$ entonces $(x, y) = (wd, \varepsilon)$ para algún $w \in \{0,1\}^*$. Luego $f_1[(x, y)] = (1wd, \varepsilon)$ es de la forma $(w'd, \varepsilon)$ con $w' = 1w \in \{0,1\}^*$, luego $f_1[(x, y)] \in \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$.

Al ser $L(A)$ el menor conjunto que cumple las condiciones anteriores, entonces necesariamente $L(A) \subseteq \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$.

Finalmente, como se tiene

$$\{(wd, \varepsilon) \mid w \in \{0,1\}^*\} \subseteq L(A) \quad \text{y} \quad L(A) \subseteq \{(wd, \varepsilon) \mid w \in \{0,1\}^*\},$$

entonces $L(A) = \{(wd, \varepsilon) \mid w \in \{0,1\}^*\}$. □

A continuación se caracteriza el lenguaje asociado al no terminal C .

Lenguaje asociado al no terminal C

El no terminal C acumula copias de la cadena generada por A . La regla

$$C \rightarrow \text{copy}(A)$$

inicializa la acumulación, mientras que

$$C \rightarrow \text{copy}(C)$$

permite concatenar nuevas copias de la primera componente sobre la segunda componente de la tupla. Por ello, C genera exactamente las tuplas cuya segunda componente consiste en una o más repeticiones de la primera. Esta idea se formaliza en el siguiente lema.

Lema 4.2. *El lenguaje generado por el símbolo no terminal C es:*

$$L(C) = \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}.$$

Demostración Primero se demostrará que para todo $n \geq 1$ si $(x, y) \in L(A)$ entonces $(x, x^n) \in L(C)$ mediante inducción sobre n .

Caso base Para $n = 1$ se tiene que $(x, y) \in L(A)$ y por el lema 4.1 $(x, y) = (x, \varepsilon)$. Como se tiene que $C \rightarrow \text{copy}(A)$ entonces

$$\text{copy}[(x, y)] = \text{copy}[(x, \varepsilon)] = (x, x) \in L(C).$$

Paso Inductivo Para $n = k$ se cumple por hipótesis de inducción que $(x, x^k) \in L(C)$ con $(x, y) \in L(A)$. Como se tiene que $C \rightarrow \text{copy}(A)$ entonces

$$\text{copy}[(x, x^k)] = (x, xx^k) = (x, x^{k+1}) \in L(C).$$

Por tanto si $(x, y) \in L(A)$ entonces $(x, x^n) \in L(C)$ para todo $n \geq 1$.

Para probar la otra inclusión

$$L(C) \subseteq \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\},$$

se procederá de forma análoga a la demostración del lema 4.1.

Por definición, $L(C)$ es el menor conjunto de tuplas en $(T^*)^2$ que cumple las siguientes condiciones:

- Si $(x, y) \in L(A)$ entonces $\text{copy}[(x, y)] \in L(C)$ por la regla $C \rightarrow \text{copy}(A)$.
- Si $(x, y) \in L(C)$ entonces $\text{copy}[(x, y)] \in L(C)$ por la regla $C \rightarrow \text{copy}(C)$.

El conjunto $\{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}$ cumple las condiciones anteriores:

- Si $(x, y) \in L(A)$ entonces por el lema 4.1

$$(x, y) = (x, \varepsilon),$$

por tanto,

$$\text{copy}[(x, y)] = \text{copy}[(x, \varepsilon)] = (x, x) \in \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}.$$

- Si $(x, y) \in \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}$ entonces

$$(x, y) = (x, x^k) \quad \text{para algun } k \geq 1,$$

$$\text{luego } \text{copy}[(x, y)] = \text{copy}[(x, x^k)] = (x, x^{k+1}) \in \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}.$$

Al ser $L(C)$ el menor conjunto que cumple las condiciones anteriores, entonces necesariamente $L(C) \subseteq \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}$.

Como se tienen las inclusiones

$$\{(x, x^n) \mid (x, y) \in L(A), n \geq 1\} \subseteq L(C) \quad \text{y} \quad L(C) \subseteq \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\},$$

entonces $L(C) = \{(x, x^n) \mid (x, y) \in L(A), n \geq 1\}$. $\square$

En la siguiente sección se caracterizará el lenguaje generado por el símbolo inicial S , es decir, el lenguaje generado por la gramática $G_{0,1,d}$ y se demostrará que coincide exactamente con el lenguaje $L_{0,1,d}$.

Lenguaje generado por S

El símbolo inicial S utiliza la función

$$\text{res}[(x, y)] = (y)$$

para extraer la segunda componente de las tuplas generadas por C . Por tanto, el lenguaje generado por la gramática coincide exactamente con el conjunto de cadenas acumuladas en dicha componente. Esta idea se formaliza en el siguiente lema.

Lema 4.3. *El lenguaje generado por el símbolo no terminal S es:*

$$L(S) = \{(y) \mid (x, y) \in L(C)\}$$

La demostración es directa dada la estructura de $G_{0,1,d}$.

Para toda $(x, y) \in L(C)$ se tiene que $y \in L(S)$ pues $y = \text{res}[(x, y)]$ y $S \rightarrow \text{res}(C)$, luego

$$\{(y) \mid (x, y) \in L(C)\} \subseteq L(S).$$

También se tiene que el conjunto $\{(y) \mid (x, y) \in L(C)\}$ cumple la condición de que si $(x, y) \in L(C)$ entonces $res[(x, y)] \in L(S)$ por la regla $S \rightarrow res(C)$. Al ser $L(S)$ el menor conjunto que cumple la condición anterior, entonces necesariamente $L(S) \subseteq \{(y) \mid (x, y) \in L(C)\}$.

Como se tiene

$$\{(y) \mid (x, y) \in L(C)\} \subseteq L(S) \quad \text{y} \quad L(S) \subseteq \{(y) \mid (x, y) \in L(C)\},$$

entonces $L(S) = \{(y) \mid (x, y) \in L(C)\}$. $\square$

Por definición, el lenguaje asociado al no terminal S es el lenguaje generado por $G_{0,1,d}$. A continuación se completa la demostración del teorema 4.1 mostrando que dicho lenguaje coincide exactamente con el lenguaje $L_{0,1,d}$.

Demostración del teorema 4.1 La demostración es una aplicación directa de los lemas 4.1, 4.2 y 4.3.

Por el lema 4.1 se tiene que el lenguaje asociado al no terminal A es el de las tuplas de la forma (wd, ε) con $w \in \{0, 1\}^*$, con este resultado y aplicando el lema 4.2 se tiene que el lenguaje asociado al no terminal C es el de las tuplas de la forma $(wd, (wd)^n)$ $w \in \{0, 1\}^*$ y $n \geq 1$. Finalmente, aplicando el lema 4.3 se tiene que el lenguaje generado por $G_{0,1,d}$ es el de las cadenas de la forma $(wd)^n$ con $w \in \{0, 1\}^*$ y $n \geq 1$, es decir, exactamente el lenguaje $L_{0,1,d}$. Por lo tanto, $L(G_{0,1,d}) = L_{0,1,d}$. $\square$

Esto demuestra que el lenguaje $L_{0,1,d}$ se puede describir con una pMCFG. Además se tiene que se puede decidir si una pMCFG es vacía en tiempo polinomial mediante el algoritmo expuesto en el capítulo 3. En la siguiente sección se muestra que estas propiedades implican que si las pMCFG fueran cerradas bajo intersección con lenguajes regulares, entonces determinar la intersección entre una pMCFG y un lenguaje regular sería un problema NP-duro.

4.2. Resultados sobre la intersección entre pMCFG y lenguajes regulares

El teorema 4.1 muestra que el lenguaje $L_{0,1,d}$ se puede describir con una pMCFG. Además en el capítulo 3 se demuestra que decidir si el lenguaje generado por una pMCFG es vacío es decidible en tiempo polinomial. Aunque en la literatura consultada no se ha encontrado ningún algoritmo correcto que determine la intersección entre una pMCFG y un lenguaje regular el teorema siguiente muestra que determinar dicha intersección es NP-duro.

Teorema 4.2. *Si las pMCFG fueran cerradas bajo intersección con lenguajes regulares, entonces determinar la intersección entre una pMCFG y un lenguaje regular es NP-duro.*

Demostración Sean R un lenguaje regular y G una pMCFG. Suponiendo que las pMCFG son cerradas bajo intersección con lenguajes regulares se tiene que existe una pMCFG G' tal que $L(G') = L(G) \cap R$. Aplicando los teoremas 4.1 y 2.5 se tiene que decidir si $L(G') = \emptyset$ es un problema NP-duro. Por el teorema 3.1 se tiene que decidir si $L(G')$ es vacío es decidable en tiempo polinomial con respecto al tamaño de G' . Por lo tanto determinar $G' = L(G) \cap R$ es un problema NP-duro. $\square$

El teorema anterior describe una consecuencia de la construcción presentada en esta sección, la cual muestra que el lenguaje $L_{0,1,d}$ se puede describir con una pMCFG. En la literatura consultada solo se encontró el algoritmo propuesto en [12] para determinar la intersección entre una MCFG el cual también se propone para las pMCFG. Sin embargo este algoritmo no es aplicable para las pMCFG, como se demuestra en el capítulo 3

El teorema anterior demuestra que dicho algoritmo no puede ser polinomial a menos que $P = NP$, lo que podría explicar la ausencia de dicho algoritmo en la literatura. Sin embargo, el teorema 4.2 no descarta la existencia de un algoritmo no polinomial para determinar la intersección entre una pMCFG y un lenguaje regular, lo cual se propone como sugerencia para futuras investigaciones.

En el siguiente capítulo se presentan las conclusiones y recomendaciones del trabajo.

Conclusiones y Recomendaciones

En este trabajo se presentó una estrategia para resolver el problema de la satisfacibilidad booleana (SAT) utilizando teoría de lenguajes formales. La solución propuesta se basa en codificar las interpretaciones de una fórmula como cadenas sobre el alfabeto $\{0, 1, d\}$ y definiendo el lenguaje $L_{\text{sat}}(F)$ de las interpretaciones que satisfacen a una fórmula dada F . A partir de este lenguaje, se puede determinar si F es satisfacible analizando si $L_{\text{sat}}(F)$ es vacío o no.

Además de definir el lenguaje $L_{\text{sat}}(F)$ se propuso una manera de construirlo mediante la intersección de una autómatata finito determinista con el lenguaje $L_{0,1,d}$. Esta construcción implica que para cualquier formalismo que genere el lenguaje $L_{0,1,d}$ y sea cerrado bajo intersección con lenguajes regulares, se tiene que construir esta intersección o el problema del vacío son problemas NP-duros. Esto, tomando como cierta la conjetura de que cualquier formalismo que genere $L_{0,1,d}$ tiene tamaño $O(1)$ en su representación.

En el capítulo 3 se demostró mediante un contraejemplo que el algoritmo reportado por la literatura para construir la intersección de una pMCFG con un lenguaje regular es incorrecto. Finalmente en el capítulo 4 se demuestra que de ser cerradas las pMCFG bajo intersección con lenguajes regulares determinar dicha intersección es un problema NP-duro, lo que implica que no existe un algoritmo polinomial para construir esta intersección a menos que $P=NP$.

La estrategia presentada constituye una forma general para resolver el problema SAT ya que no depende del orden de las variables. Este acercamiento puede contribuir a nuevas formas de abordar el problema de la satisfacibilidad booleana y otros problemas relacionados en la teoría de la complejidad y la teoría de lenguajes formales.

A partir del trabajo realizado se proponen como temas de investigación futura los siguientes:

- Buscar formalismos que generen el lenguaje $L_{0,1,d}$ y sean cerrados bajo intersección con lenguajes regulares. Analizar el problema del vacío para el formalismo que se obtiene luego de la intersección. En la literatura consultada para la realización de este trabajo se encontraron las gramáticas de índice global [4], las cuales cumplen las propiedades anteriores.

- Demostrar que cualquier formalismo que genere el lenguaje $L_{0,1,d}$ tiene tamaño $O(1)$ en su representación.
- Demostrar que las gramáticas libres de contexto múltiple son cerradas bajo intersección con lenguajes regulares.
- Analizar la construcción efectuada para construir $L_{\text{sat}}(F)$ en el capítulo 2 para el caso en el que la fórmula F es una fórmula 3-CNF. En este caso, cada cláusula tiene exactamente tres literales, lo que podría simplificar la construcción y permitir obtener resultados más específicos sobre la complejidad de la construcción.

Referencias

- [1] Alina Fernández Arias. «El problema de la satisfacibilidad booleana libre del contexto». En: (2007) (vid. págs. 1, 13).
- [2] A. Biere, M. Heule y H. van Maaren. *Handbook of Satisfiability: Second Edition*. Frontiers in Artificial Intelligence and Applications. IOS Press, 2021, págs. 3-55. ISBN: 9781643681610. URL: <https://books.google.com/books?id=dUAvEAAAQBAJ> (vid. pág. 1).
- [3] Pierre Boullier. *A Cubic Time Extension of Context-Free Grammars*. Research Report RR-3611. INRIA, 1999. URL: <https://inria.hal.science/inria-00073067> (vid. págs. 28, 29).
- [4] José M. Castaño. «Global Index Languages». Tesis doct. The Faculty of the Graduate School of Arts y Sciences Brandeis University, 2004 (vid. págs. 28, 29, 53).
- [5] Stephen A. Cook. «The Complexity of Theorem-Proving Procedures». En: *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1971, págs. 151-158. DOI: 10.1145/800157.805047 (vid. pág. 13).
- [6] John E. Hopcroft, Rajeev Motwani y Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd. Addison-Wesley, 2006. ISBN: 9780321455369 (vid. págs. 1, 2, 6-11, 13, 24, 25, 35, 38).
- [7] Tim Hunter. «The Chomsky Hierarchy». En: *Linguistic Theory Journal* 35.2 (2020), págs. 123-145. URL: <https://www.timhunter.humspace.ucla.edu/papers/blackwell-chomsky-hierarchy.pdf> (vid. pág. 8).
- [8] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Springer, ene. de 2010. ISBN: 978-3-642-14845-3. DOI: 10.1007/978-3-642-14846-0 (vid. pág. 35).
- [9] Manuel Aguilera López. «Problema de la Satisfacibilidad Booleana de Concatenación de Rango Simple». En: (2016) (vid. págs. 1, 13, 14).
- [10] Raudel Alejandro Gómez Molina. «Una aproximación al lenguaje de todas las fórmulas booleanas satisfacibles». En: (2025) (vid. págs. 13, 14, 28).

- [11] José Jorge Rodríguez Salado. «Gramáticas Matriciales Simples. Primera aproximación para una solución al problema SAT». En: (2019) (vid. págs. 1, 13, 14).
- [12] Hiroyuki Seki et al. «On Multiple Context-Free Grammars». En: *Theor. Comput. Sci.* 88 (1991), págs. 191-229. URL: <https://api.semanticscholar.org/CorpusID:34940133> (vid. págs. 2, 28-31, 33-35, 38, 40, 41, 52).